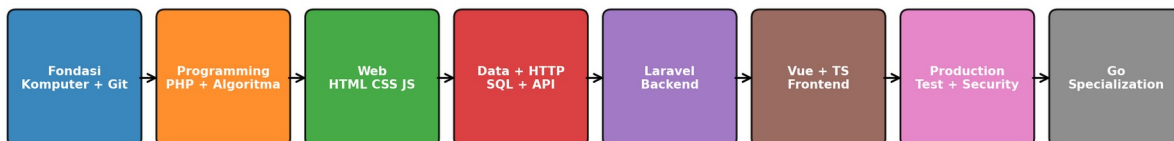


# BUKU LENGKAP FULL-STACK DEVELOPER 2026

Fondasi - PHP - Laravel - Vue 3 - TypeScript - Testing - Security - Deployment - Docker  
- CI/CD - Go - AI-assisted Development

## Peta Belajar Full-Stack 2026

Belajar berurutan: pahami konsep -> praktik -> proyek -> test -> deploy



Edisi pembelajaran terpadu - 25 September 2026

Disusun sebagai perluasan penuh dari Roadmap Belajar Full-Stack Developer 2026 yang kamu lampirkan. Fokusnya bukan hafalan framework, tetapi kemampuan membangun, menguji, mengamankan, men-deploy, dan memelihara aplikasi nyata.

# PETUNJUK PENGGUNAAN

Cara belajar yang membuat materi menempel dan berubah menjadi kemampuan nyata

## Cara menggunakan buku ini

Buku ini mempertahankan delapan bagian utama dari roadmap awal dan memperluas setiap topik menjadi materi teori, mental model, contoh kode, latihan, kesalahan umum, checkpoint, dan proyek. Bagian tambahan 2026 membahas AI-assisted development, software supply chain, privasi, cloud, dan cara bekerja dengan dokumentasi digital modern.

### Aturan utama

Jangan naik level hanya karena selesai membaca. Naik setelah bisa menjelaskan konsep dengan kata sendiri, membuat contoh dari nol, mengubah requirement, menguji kasus gagal, dan menyimpan hasilnya di Git.

| Tahap         | Aktivitas                     | Bukti selesai                         |
|---------------|-------------------------------|---------------------------------------|
| 1. Pahami     | Baca mental model dan istilah | Bisa menjelaskan tanpa menyalin       |
| 2. Ikuti      | Tulis contoh kecil            | Contoh berjalan dan kamu tahu mengapa |
| 3. Modifikasi | Ubah requirement              | Tidak bergantung pada tutorial        |
| 4. Bangun     | Mini project                  | Fitur selesai dari nol                |
| 5. Uji        | Kasus gagal + test            | Bug penting tertangkap                |
| 6. Refleksi   | README + catatan keputusan    | Bisa menjelaskan trade-off            |

## Acuan teknologi per 25 September 2026

| Teknologi  | Acuan belajar           | Catatan                                        |
|------------|-------------------------|------------------------------------------------|
| PHP        | 8.5.11                  | Current stable; gunakan patch security terbaru |
| Laravel    | 13.x                    | Mendukung PHP 8.3-8.5                          |
| Vue        | Vue 3                   | Composition API dan Single File Component      |
| TypeScript | Current stable          | Pelajari setelah JavaScript                    |
| Go         | 1.27.x                  | Go 1.27 rilis Agustus 2026                     |
| Node.js    | LTS yang masih didukung | Untuk npm/Vite/tooling; prioritaskan LTS       |
| Database   | MySQL atau PostgreSQL   | Konsep SQL lebih penting dari vendor           |

### Prinsip versi

Versi framework akan berubah. Konsep HTTP, SQL, data modeling, authorization, testing, security, observability, dan problem solving bertahan lebih lama. Gunakan dokumentasi versi proyek yang sedang kamu kerjakan.

## Daftar isi terpadu

- Bagian I - Fondasi developer dan cara berpikir (1-5)
- Bagian II - Web fundamental (6-11)
- Bagian III - Data, PHP web, dan Laravel (12-16)
- Bagian IV - Frontend modern: Vite, TypeScript, Vue, Inertia (17-21)

- Bagian V - Quality engineering: testing, security, performance (22-25)
- Bagian VI - Production: Linux, deployment, Docker, CI/CD, backup (26-31)
- Bagian VII - Go untuk backend specialization (32-35)
- Bagian VIII - Proyek terpadu, roadmap 26 minggu, debugging, portfolio (36-44)
- Bagian IX - Materi era digital 2026: AI, supply chain, cloud, privacy, API, kerja kolaboratif (45-51)
- Lampiran - Cheat sheet, latihan, rubric, glossary, referensi resmi

# BAGIAN I

Fondasi developer dan cara berpikir

## Git Workflow Sehat

Setiap perubahan punya tujuan, bukti verifikasi, dan riwayat yang bisa dibaca.



# 1. Lingkungan kerja developer

Framework tidak dapat menggantikan pemahaman dasar komputer. Banyak bug pemula sebenarnya adalah masalah path, process, port, permission, environment, atau toolchain.

## Tujuan belajar

- ☐ Memahami file/folder, path, process, service, port, localhost, IP, DNS dasar.
- ☐ Menggunakan terminal dan editor dengan nyaman.
- ☐ Memisahkan source code dari konfigurasi dan secret.
- ☐ Membaca error message serta stack trace secara sistematis.

## Mental model

Ketika aplikasi gagal, tanyakan berurutan: file yang benar? program yang benar? process hidup? port benar? konfigurasi terbaca? dependency tersedia? baru kemudian curigai kode bisnis.

### Inti konsep

Ketika aplikasi gagal, tanyakan berurutan: file yang benar? program yang benar? process hidup? port benar? konfigurasi terbaca? dependency tersedia? baru kemudian curigai kode bisnis.

## Materi inti

### File system

Absolute path menunjuk lokasi lengkap; relative path dihitung dari working directory. Extension membantu mengenali format, sedangkan permission menentukan siapa yang boleh membaca, menulis, atau mengeksekusi.

### Process, service, port

Program adalah kode di disk; process adalah program yang sedang berjalan. Service adalah proses jangka panjang. Port mengidentifikasi endpoint jaringan pada suatu host.

### Environment

Environment variable cocok untuk konfigurasi yang berbeda antar mesin. File .env memudahkan development, tetapi secret production sebaiknya dikelola oleh secret store atau environment platform.

### Toolchain

VS Code, terminal, Git, PHP, Composer, Node.js/npm, database, dan browser DevTools adalah rantai kerja. Kenali versi dan lokasi executable agar masalah PATH mudah didiagnosis.

## Contoh kode lengkap

### BASH

```
# Git Bash / Linux/macOS
pwd
ls -la
mkdir -p belajar-programming/{latihan,notes,projects}
cd belajar-programming
printf "# Catatan belajar\n" > README.md

# Cek toolchain
```

```
php -v
composer --version
node -v
npm -v
git --version
```

## Kesalahan umum

- Menjalankan command dari folder yang salah.
- Mencampur config, secret, dan source code.
- Menghapus folder dengan rm tanpa memastikan path.
- Menginstal banyak extension editor tanpa memahami fungsinya.

## Latihan praktik

1. Buat struktur folder belajar hanya dari terminal.
2. Jalankan server lokal sederhana dan akses dari browser.
3. Sengaja gunakan port yang sedang dipakai lalu baca error-nya.
4. Buat .env.example tanpa secret dan .env lokal yang di-ignore Git.

### Checkpoint kompetensi

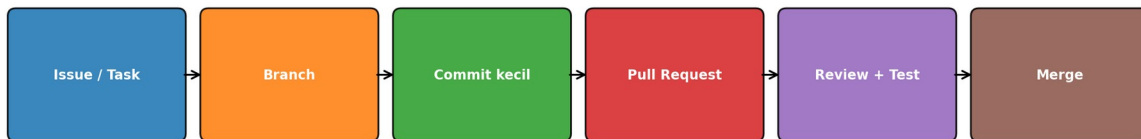
Bisa menjelaskan perbedaan program/process/service, localhost/IP, port, absolute/relative path, serta menemukan penyebab umum command not found atau port already in use.

## 2. Git dan GitHub

Git menyimpan sejarah perubahan; GitHub menyediakan remote hosting, review, issue, automation, dan kolaborasi. Git bukan sekadar tempat upload ZIP.

### Git Workflow Sehat

Setiap perubahan punya tujuan, bukti verifikasi, dan riwayat yang bisa dibaca.



### Tujuan belajar

- ☐ Memahami working tree, staging area, commit, branch, remote, merge.
- ☐ Membuat commit kecil yang bermakna.
- ☐ Mengatasi conflict dasar tanpa kehilangan perubahan.
- ☐ Menggunakan pull request sebagai mekanisme review.

### Mental model

Setiap commit seharusnya menjawab satu pertanyaan: perubahan logis apa yang saya buat dan mengapa?

#### Inti konsep

Setiap commit seharusnya menjawab satu pertanyaan: perubahan logis apa yang saya buat dan mengapa?

### Materi inti

#### Tiga area Git

Working tree adalah file saat ini; staging area adalah snapshot yang akan masuk commit; repository menyimpan commit yang sudah dibuat.

#### Branch

Branch adalah pointer ke rangkaian commit. Feature branch menjaga perubahan terisolasi sampai siap direview.

#### Remote

fetch mengambil informasi remote tanpa merge; pull umumnya fetch + integrasi; push mengirim commit lokal ke remote.

## Conflict

Conflict bukan kerusakan. Itu tanda Git tidak dapat menentukan hasil otomatis. Baca kedua sisi, pilih hasil yang benar, test, lalu commit.

## Contoh kode lengkap

### BASH

```
git init
git status
git add .
git commit -m "Initial learning workspace"

git switch -c feature/login-form
# edit files
git diff
git add src/login.php
git commit -m "Add login form validation"

git switch main
git merge feature/login-form
git log --oneline --graph --decorate
```

## Kesalahan umum

- Commit satu hari penuh sekaligus.
- Menaruh password/token di repository.
- Menggunakan git push --force tanpa memahami dampak.
- Merge tanpa membaca diff atau menjalankan test.

## Latihan praktik

1. Buat repo latihan dan minimal 10 commit kecil.
2. Buat dua branch yang mengubah baris sama agar conflict terjadi, lalu selesaikan.
3. Buat .gitignore untuk .env, vendor/node\_modules sesuai konteks.
4. Tulis pull request description: masalah, solusi, cara test, risiko.

### Checkpoint kompetensi

Bisa menjelaskan status file untracked/modified/staged, membuat branch, merge, menyelesaikan conflict, dan membaca git log/diff tanpa GUI.



## 3. Fundamental programming dengan PHP

PHP dipakai sebagai bahasa pertama agar fondasi programming langsung tersambung ke backend dan Laravel. Tujuannya memahami program tanpa framework.

### Tujuan belajar

- ☐ Menguasai variabel, tipe data, operator, kondisi, loop, function, array, error.
- ☐ Memahami input-process-output.
- ☐ Menulis fungsi dengan parameter dan return type yang jelas.
- ☐ Menggunakan exception untuk kondisi gagal yang memang perlu dihentikan.

### Mental model

Setiap fitur dapat dibongkar menjadi input -> validasi -> proses -> output. Saat debugging, verifikasi nilai pada setiap tahap.

#### Inti konsep

Setiap fitur dapat dibongkar menjadi input -> validasi -> proses -> output. Saat debugging, verifikasi nilai pada setiap tahap.

### Materi inti

#### Data dan tipe

Integer, float, string, boolean, null, array, object. Type declaration membantu kontrak fungsi tetapi tidak menghapus kebutuhan validasi input eksternal.

#### Control flow

if/match memilih jalur; loop mengulangi pekerjaan. Hindari kondisi bertingkat terlalu dalam dengan early return dan function kecil.

#### Function

Function yang baik memiliki tujuan jelas, input terdefinisi, output terprediksi, dan efek samping seminimal mungkin.

#### Error dan exception

Bedakan data invalid yang dapat dijelaskan ke user dari bug internal. Tangkap exception di boundary yang bisa melakukan recovery atau mengubahnya menjadi response aman.

### Contoh kode lengkap

#### PHP

```
<?php
declare(strict_types=1);

function hitungTotal(int $harga, int $jumlah, float $diskon = 0): int
{
    if ($harga < 0 || $jumlah < 0) {
        throw new InvalidArgumentException('Nilai tidak boleh negatif');
    }

    $subtotal = $harga * $jumlah;
```

```

    $potongan = (int) round($subtotal * $diskon);
    return max(0, $subtotal - $potongan);
}

$items = [
    ['nama' => 'Kopi', 'harga' => 15000, 'jumlah' => 2],
    ['nama' => 'Teh', 'harga' => 10000, 'jumlah' => 1],
];

$total = 0;
foreach ($items as $item) {
    $total += hitungTotal($item['harga'], $item['jumlah']);
}

echo "Total: Rp" . number_format($total, 0, ',', '.') . PHP_EOL;

```

## Kesalahan umum

- Menghafal sintaks tanpa memahami nilai yang masuk/keluar.
- Menggunakan global variable untuk semua hal.
- Mencampur input/output dengan logic sehingga sulit dites.
- Menangkap semua exception lalu mengabaikannya.

## Latihan praktik

1. Buat kalkulator CLI dengan validasi pembagian nol.
2. Buat program daftar nilai: rata-rata, minimum, maksimum, status lulus.
3. Buat pencarian sederhana pada array produk.
4. Baca file JSON, ubah satu data, lalu tulis kembali dengan format valid.

### Checkpoint kompetensi

Bisa membuat program CLI kecil dari nol tanpa framework dan menjelaskan alur data, function, kondisi gagal, serta cara memperbaiki bug sederhana.

## 4. Algoritma, problem solving, dan struktur data

Developer aplikasi tidak harus memulai dari algoritma kompetitif. Yang lebih penting adalah memecah requirement, memilih representasi data, dan memahami biaya operasi secara intuitif.

### Tujuan belajar

- ☐ Menulis pseudocode.
- ☐ Menerapkan decomposition, sequence, selection, iteration.
- ☐ Memahami array/list, map, set, stack, queue, tree, graph.
- ☐ Memiliki intuisi  $O(1)$ ,  $O(\log n)$ ,  $O(n)$ ,  $O(n^2)$ .

### Mental model

Sebelum menulis kode, tulis apa yang diketahui, apa yang harus dihasilkan, aturan, contoh normal, dan kasus batas.

#### Inti konsep

Sebelum menulis kode, tulis apa yang diketahui, apa yang harus dihasilkan, aturan, contoh normal, dan kasus batas.

### Materi inti

#### Decomposition

Ubah requirement besar menjadi input, aturan, penyimpanan, output, dan edge case. Pecah lagi sampai tiap langkah dapat diimplementasikan dan diuji.

#### Search dan sort

Linear search cocok untuk data kecil/tidak terurut. Binary search membutuhkan urutan dan random access. Sorting biasanya gunakan fungsi library kecuali tujuan belajar.

#### Data structure

Map cocok mencari berdasarkan key; set memastikan unik; queue menjaga urutan FIFO; stack LIFO; tree mewakili hierarki; graph mewakili hubungan umum.

#### Complexity

Big-O menggambarkan pertumbuhan kerja saat input membesar. Untuk aplikasi, fokus pada operasi yang ada di hot path: query, loop besar, serialization, network.

### Contoh kode lengkap

#### TEXT

```
PSEUDOCODE: proses pengajuan layanan
INPUT data_warga, layanan, lampiran
VALIDATE identitas
VALIDATE layanan tersedia
VALIDATE lampiran sesuai aturan
IF valid THEN
  buat tiket unik
  simpan pengajuan
  catat audit event
OUTPUT tiket + status pending
```

```
ELSE
    OUTPUT daftar error
END
```

## Latihan praktik

1. Tulis pseudocode login yang aman dari sisi alur (tanpa detail cryptography).
2. Bandingkan pencarian array linear dengan map berdasarkan ID.
3. Implementasikan queue sederhana dari array untuk memahami FIFO.
4. Ambil fitur "pengaduan warga" dan pecah menjadi minimal 8 submasalah.

### Checkpoint kompetensi

Bisa mengubah requirement teks menjadi pseudocode, memilih struktur data yang masuk akal, dan menjelaskan mengapa solusi  $O(n^2)$  dapat bermasalah pada data besar.

## 5. OOP dan clean code dasar

OOP membantu memodelkan data dan perilaku, tetapi class bukan tujuan. Clean code berarti kode yang mudah dipahami, diuji, diubah, dan aman - bukan sekadar pendek.

### Tujuan belajar

- ☐ Memahami class, object, property, method, constructor.
- ☐ Memahami encapsulation, interface, polymorphism, composition.
- ☐ Menggunakan single responsibility secara pragmatis.
- ☐ Refactor setelah perilaku dilindungi test/verifikasi.

### Materi inti

#### Encapsulation

Objek menjaga invariants internal; jangan memaksa caller mengatur detail yang seharusnya menjadi tanggung jawab object.

#### Interface

Interface menyatakan kemampuan/kontrak. Gunakan ketika ada variasi implementasi atau boundary yang perlu dites, bukan otomatis untuk setiap class.

#### Composition

Menggabungkan object sering lebih fleksibel daripada inheritance dalam aplikasi bisnis.

#### Clean code

Nama yang jelas, function kecil, dependency eksplisit, error handling konsisten, serta struktur yang mengikuti domain lebih bernilai daripada pola abstraksi berlebihan.

### Contoh kode lengkap

PHP

```
<?php
interface Notifier
{
    public function send(string $recipient, string $message): void;
}

final class ServiceRequestManager
{
    public function __construct(private Notifier $notifier) {}

    public function complete(string $email, string $ticket): void
    {
        $this->notifier->send($email, "Pengajuan {$ticket} selesai");
    }
}
```

### Kesalahan umum

- Membuat class untuk setiap kata benda tanpa manfaat.
- Inheritance panjang hanya untuk reuse.

- God class yang mengurus request, query, email, dan rendering sekaligus.
- Refactor besar tanpa test atau baseline perilaku.

## Latihan praktik

1. Refactor satu script procedural menjadi beberapa function, lalu class hanya jika ada tanggung jawab yang jelas.
2. Buat interface Storage dan dua implementasi sederhana in-memory/file untuk latihan dependency injection.
3. Cari duplikasi nyata di latihan lama dan refactor tanpa mengubah output.

### Checkpoint kompetensi

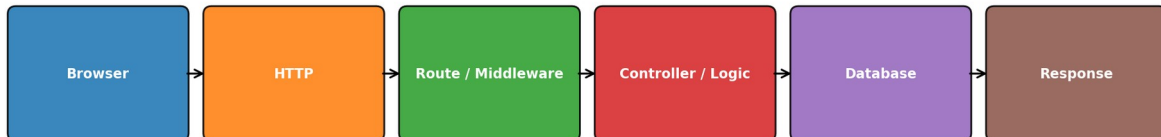
Bisa menjelaskan kapan memakai function, class, interface, composition, dan dependency injection dengan contoh nyata - bukan definisi hafalan.

# BAGIAN II

Web fundamental: HTML, CSS, JavaScript, browser, HTTP, accessibility, SEO

## Mental Model Aplikasi Web

Debug dari kiri ke kanan; verifikasi input dan output tiap lapisan.



## 6. HTML semantik

HTML mendefinisikan struktur dan makna dokumen. Halaman yang baik tetap mempunyai urutan masuk akal tanpa CSS dan dapat dinavigasi dengan keyboard serta teknologi bantu.

### Tujuan belajar

- ☐ Menulis struktur dokumen valid.
- ☐ Menggunakan semantic landmarks dan heading hierarchy.
- ☐ Membuat form dengan label dan tipe input yang tepat.
- ☐ Memahami alt text dan SEO dasar.

### Materi inti

#### Dokumen

Gunakan doctype, lang, charset, viewport, title. Satu h1 utama per konteks halaman adalah pola sederhana yang mudah dirawat.

#### Semantik

header/nav/main/section/article/aside/footer memberi makna. Gunakan button untuk aksi dan a untuk navigasi.

#### Form

label harus terhubung dengan input. Native input types memberikan keyboard dan validasi dasar yang lebih baik, tetapi server-side validation tetap wajib.

#### Media

Alt text menjelaskan fungsi/konten gambar yang relevan; gambar dekoratif dapat memakai alt kosong.

### Contoh kode lengkap

#### HTML

```
<!doctype html>
<html lang="id">
<head>
  <meta charset="utf-8">
  <meta name="viewport" content="width=device-width,initial-scale=1">
  <title>Portal Gampong</title>
  <meta name="description" content="Informasi dan layanan gampong">
</head>
<body>
<header>
  <nav aria-label="Navigasi utama">
    <a href="/">Beranda</a>
    <a href="/layanan">Layanan</a>
  </nav>
</header>
<main>
  <article>
    <h1>Berita Gampong</h1>
    <p>Informasi terbaru untuk warga.</p>
  </article>
</main>
<footer>Kontak kantor</footer>
```



```
</body>
</html>
```

## Latihan praktik

1. Buat homepage portal gampong hanya HTML semantik.
2. Audit heading dari h1 sampai h3 dan perbaiki lompatan yang tidak logis.
3. Buat form pengaduan dengan label, required, textarea, select, file input.

### Checkpoint kompetensi

Halaman dapat dipahami tanpa CSS, semua control form memiliki label, link dan button digunakan sesuai fungsi, dan urutan heading logis.

## 7. CSS modern dan responsive design

CSS adalah sistem layout dan presentasi yang responsif terhadap ukuran layar, zoom, panjang teks, preferensi user, dan keadaan UI.

### Tujuan belajar

- ☐ Memahami cascade, specificity, inheritance, box model.
- ☐ Menguasai Flexbox dan Grid.
- ☐ Menerapkan mobile-first, media/container query sesuai kebutuhan.
- ☐ Membuat dark mode, focus state, reduced motion, dan layout tahan teks panjang.

### Materi inti

#### Cascade

Urutan source, specificity, importance, dan inheritance menentukan style akhir. Lebih baik struktur selector sederhana daripada perang !important.

#### Layout

Flexbox ideal untuk satu dimensi; Grid kuat untuk dua dimensi. Gunakan gap, minmax(), auto-fit, dan min-width:0 untuk layout yang tahan konten.

#### Responsive

Mulai dari layout kecil, lalu tambah ruang. Uji bukan hanya device preset, tetapi juga zoom 200%, font besar, dan teks panjang.

#### Design tokens

CSS custom properties membuat warna, spacing, radius, dan ukuran konsisten. Gunakan token semantik seperti --surface, --text, --accent.

### Contoh kode lengkap

#### css

```
/* tokens */
:root {
  --space-1: .5rem;
  --space-2: 1rem;
  --radius: .75rem;
  --surface: #fff;
  --text: #18212b;
}

* { box-sizing: border-box; }
body { margin: 0; font: 1rem/1.6 system-ui, sans-serif; color: var(--text); background: var(--surface); }
.grid { display: grid; grid-template-columns: repeat(auto-fit,minmax(min(100%,16rem),1fr)); gap: var(--space-2); }
.card { min-width: 0; padding: var(--space-2); border: 1px solid currentColor; border-radius: var(--radius); }
:focus-visible { outline: 3px solid currentColor; outline-offset: 3px; }
@media (prefers-reduced-motion: reduce) { *,*::before,*::after { animation-duration: .01ms !important; transition-duration: .01ms !important; } }
```

## Kesalahan umum

- Fixed width yang menyebabkan horizontal overflow.
- Menyembunyikan focus outline tanpa pengganti.
- Menggunakan absolute positioning untuk layout utama.
- Warna teks kontras rendah demi estetika.

## Latihan praktik

1. Bangun layout 1 kolom mobile menjadi 3 kolom desktop tanpa fixed width.
2. Uji judul 120 karakter, zoom 200%, dan bahasa dengan kata panjang.
3. Buat light/dark mode berbasis data-theme atau prefers-color-scheme.

### Checkpoint kompetensi

UI tetap terbaca pada mobile, desktop, zoom 200%, dark mode, reduced motion, serta tidak rusak oleh teks panjang atau data kosong.

## 8. JavaScript fundamental

JavaScript adalah runtime utama browser. Vue dan TypeScript akan jauh lebih mudah jika function, object, array, scope, event, Promise, dan async/await sudah dipahami.

### Tujuan belajar

- ☐ Menguasai let/const, primitive, object, array, function, scope.
- ☐ Menggunakan array methods secara tepat.
- ☐ Memahami modules, Promise, async/await, error handling.
- ☐ Menggunakan fetch dan JSON tanpa framework.

### Materi inti

#### Runtime

TypeScript akan dihapus menjadi JavaScript saat build. Karena itu behavior runtime tetap mengikuti JavaScript.

#### Array methods

map mengubah item, filter memilih, find mencari satu, some/every mengevaluasi kondisi, reduce mengakumulasi. Jangan memakai metode hanya demi terlihat fungsional.

#### Async

Promise mewakili hasil di masa depan. await menghentikan function async sampai Promise selesai tanpa memblokir event loop browser.

#### Modules

import/export membuat dependency eksplisit dan memudahkan bundler/tree-shaking.

### Contoh kode lengkap

#### JAVASCRIPT

```
export async function loadPosts(signal) {
  const response = await fetch('/api/posts', {
    headers: { Accept: 'application/json' },
    signal,
  });

  if (!response.ok) {
    throw new Error(`HTTP ${response.status}`);
  }

  const payload = await response.json();
  return payload.data ?? payload;
}

const titles = (await loadPosts()).map(post => post.title);
console.log(titles);
```

### Kesalahan umum

- Menggunakan var tanpa kebutuhan.
- Tidak memeriksa response.ok pada fetch.

- Mencampur DOM query, network, dan business logic dalam satu function besar.
- Menyimpan secret di JavaScript frontend.

## Latihan praktik

1. Buat filter berita client-side dari array.
2. Ambil JSON dari endpoint lokal lalu render list.
3. Tangani loading, success, empty, error.
4. Gunakan AbortController agar request dapat dibatalkan.

### Checkpoint kompetensi

Bisa membuat fitur interaktif tanpa Vue: event, DOM, form, fetch, async/await, dan error state.

## 9. Browser, DOM, DevTools, dan debugging

Browser adalah runtime lengkap: DOM, CSSOM, network, storage, JavaScript engine, cache, security model, dan DevTools. Developer yang bisa membaca DevTools lebih cepat menemukan bug daripada menebak-nebak.

### Tujuan belajar

- ☐ Memahami DOM tree dan event propagation.
- ☐ Menggunakan Network, Console, Sources, Application, Performance.
- ☐ Membedakan bug UI, network, server, cache, CORS.
- ☐ Melakukan debugging hypothesis-driven.

### Mental model

Reproduksi -> observasi -> lokalisasi lapisan -> hipotesis kecil -> satu perubahan -> verifikasi -> regression test.

#### Inti konsep

Reproduksi -> observasi -> lokalisasi lapisan -> hipotesis kecil -> satu perubahan -> verifikasi -> regression test.

### Materi inti

#### DOM

DOM adalah representasi objek dari dokumen. Event umumnya berjalan capture -> target -> bubble; delegation memanfaatkan bubbling untuk banyak elemen dinamis.

#### Network

Periksa URL, method, status, request payload, response body, headers, timing. Banyak "bug frontend" sebenarnya response 401/422/500.

#### Storage

Cookie, localStorage, sessionStorage memiliki karakteristik berbeda. Jangan menyimpan credential atau data sensitif sembarangan.

#### Debugger

Breakpoint dan step execution lebih akurat daripada menambah console.log di seluruh aplikasi.

### Latihan praktik

1. Debug request 404 dan 422 menggunakan Network tab.
2. Pasang breakpoint pada submit handler dan inspeksi scope.
3. Hapus cache/storage dan amati perubahan.
4. Simulasikan koneksi lambat untuk menguji loading state.

#### Checkpoint kompetensi

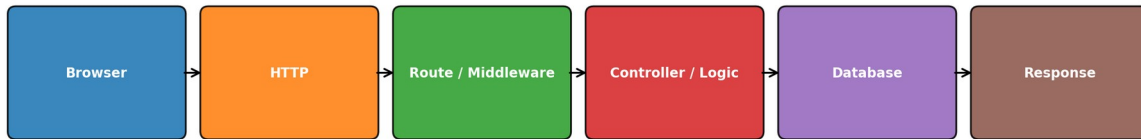
Bisa menemukan apakah masalah terjadi sebelum request, saat request, di backend, atau saat rendering response tanpa menebak secara acak.

# 10. HTTP, URL, JSON, cookie, session, CORS, caching

Aplikasi web adalah pertukaran request-response. Memahami HTTP membuat routing, API, authentication, browser security, dan debugging jauh lebih masuk akal.

## Mental Model Aplikasi Web

Debug dari kiri ke kanan; verifikasi input dan output tiap lapisan.



## Tujuan belajar

- ☐ Membaca URL dan HTTP message.
- ☐ Memahami method, status, headers, body, content negotiation.
- ☐ Membedakan cookie, session, token.
- ☐ Memahami CORS, TLS, cache secara konseptual.

## Materi inti

### URL

scheme://host:port/path?query#fragment. Fragment tidak dikirim ke server dalam request HTTP biasa.

### Method

GET membaca, POST membuat/menjalankan aksi, PUT/PATCH memperbarui, DELETE menghapus. Semantics membantu cache, retry, monitoring, dan desain API.

### Status

2xx berhasil, 3xx redirect/cache, 4xx masalah request/izin, 5xx masalah server. Gunakan status yang mencerminkan hasil.

### Cookie/session

Session biasanya menyimpan state di server dan cookie menyimpan session identifier. Gunakan HttpOnly, Secure, SameSite sesuai kebutuhan.

### CORS

CORS adalah aturan browser untuk cross-origin access; bukan mekanisme authentication. Server tetap harus memverifikasi identitas dan izin.



## Caching

Cache-Control/ETag membantu mengurangi bandwidth dan latency. Jangan cache response private tanpa strategi yang tepat.

## Contoh kode lengkap

### HTTP

```
GET /api/posts?page=2 HTTP/1.1
Host: example.test
Accept: application/json
```

```
HTTP/1.1 200 OK
Content-Type: application/json
Cache-Control: private, max-age=60
```

```
{"data":[{"id":1,"title":"Berita"},"meta":{"page":2}]}
```

## Latihan praktik

1. Gunakan DevTools untuk menemukan request, headers, response, timing.
2. Buat tabel contoh kapan memakai 200, 201, 204, 401, 403, 404, 422, 429, 500.
3. Jelaskan beda authentication dan CORS.

### Checkpoint kompetensi

Bisa membaca request/response HTTP dan menjelaskan mengapa suatu fitur menghasilkan status tertentu.

# 11. Accessibility, SEO, performance, dan UX engineering

Website yang berhasil tidak hanya "bagus di layar saya". Ia harus dapat digunakan, ditemukan, cepat, dan memberikan state yang jelas saat data loading, kosong, gagal, atau berhasil.

## Tujuan belajar

- ☐ Menerapkan keyboard navigation dan focus state.
- ☐ Menerapkan SEO teknis dasar.
- ☐ Memahami Core Web Vitals secara konseptual.
- ☐ Mendesain loading/empty/error/success state.

## Materi inti

### Accessibility

Gunakan HTML native lebih dulu. ARIA melengkapi ketika elemen native tidak cukup. Uji dengan keyboard dan zoom, bukan hanya mouse.

### SEO

Title, description, heading, canonical, crawlable content, sitemap, robots, Open Graph, structured data bila relevan.

### Performance

Optimalkan gambar, ukuran JS, caching, layout stability, dan latency backend berdasarkan pengukuran.

### UX states

Setiap data-driven UI perlu state loading, empty, error, success. Tombol submit harus mencegah duplicate action dan memberi feedback.

## Kesalahan umum

- Menggunakan div clickable tanpa keyboard semantics.
- Memasang lazy loading pada hero/LCP tanpa pertimbangan.
- Menjejalkan semua SEO keyword secara tidak natural.
- Menghilangkan loading state sehingga user mengklik berulang.

## Latihan praktik

1. Navigasikan halaman hanya dengan Tab/Shift+Tab/Enter.
2. Audit alt text dan label form.
3. Ukur performa awal dengan browser tools lalu optimalkan satu bottleneck terukur.
4. Buat empty/error state yang informatif untuk daftar berita.

### Checkpoint kompetensi

Satu halaman checkpoint dapat dipakai keyboard, responsif, punya metadata dasar, state UI lengkap, dan tidak memiliki masalah performa yang jelas pada pengukuran awal.

# BAGIAN III

Data, PHP web, Composer, Laravel, arsitektur backend

## Laravel Request Lifecycle

Tambahkan lapisan hanya ketika kompleksitas benar-benar membutuhkannya.



## 12. Database relasional dan SQL

Database relasional menyimpan fakta dalam tabel yang dihubungkan constraint. ORM berguna, tetapi SQL tetap perlu dipahami agar query benar, cepat, dan konsisten.

### Tujuan belajar

- ☐ Mendesain table, key, constraint, index.
- ☐ Menulis CRUD, filter, aggregate, JOIN, transaction.
- ☐ Memahami normalisasi dan N+1 query.
- ☐ Menggunakan EXPLAIN secara konseptual untuk membaca query plan.

### Materi inti

#### Schema

Pilih tipe data yang mencerminkan domain. NOT NULL, UNIQUE, FOREIGN KEY adalah aturan data, bukan dekorasi.

#### Index

Index mempercepat lookup tetapi menambah storage dan biaya write. Index harus mengikuti pola query nyata.

#### Join

INNER JOIN mengembalikan baris yang cocok; LEFT JOIN mempertahankan seluruh sisi kiri. Hindari query loop satu per item jika bisa eager-load/join.

#### Transaction

Sekumpulan perubahan yang harus berhasil/gagal bersama perlu transaction. Keep transaction short dan tangani retry bila database mengharuskan.

#### Normalization

1NF-3NF membantu mengurangi duplikasi dan anomaly. Denormalisasi boleh jika ada alasan performa yang diukur dan konsistensi dijaga.

### Contoh kode lengkap

#### SQL

```
CREATE TABLE posts (
  id BIGINT PRIMARY KEY AUTO_INCREMENT,
  user_id BIGINT NOT NULL,
  title VARCHAR(200) NOT NULL,
  slug VARCHAR(220) NOT NULL UNIQUE,
  body TEXT NOT NULL,
  published_at DATETIME NULL,
  created_at DATETIME NOT NULL,
  FOREIGN KEY (user_id) REFERENCES users(id),
  INDEX idx_posts_published_at (published_at)
);

SELECT u.name, COUNT(p.id) AS total_posts
FROM users u
LEFT JOIN posts p ON p.user_id = u.id
```

```
GROUP BY u.id, u.name
ORDER BY total_posts DESC;
```

## Latihan praktik

1. Desain schema berita, kategori, user, komentar.
2. Tulis JOIN untuk menampilkan berita + penulis + kategori.
3. Buat transaction simulasi pembayaran/stock sederhana.
4. Bandingkan query dengan dan tanpa index menggunakan EXPLAIN di database lokal.

### Checkpoint kompetensi

Bisa mendesain schema dari requirement, menulis JOIN/aggregate, menjelaskan constraint, transaction, index, dan mengenali pola N+1.

## 13. PHP untuk web tanpa framework

Sebelum Laravel, bangun satu aplikasi PHP sederhana agar request, validation, session, PDO, redirect, upload, dan rendering tidak menjadi "magic" framework.

### Tujuan belajar

- ☐ Memproses GET/POST secara aman.
- ☐ Menggunakan PDO prepared statements.
- ☐ Membuat session login dan password hashing.
- ☐ Menerapkan server-side validation dan upload aman.

### Materi inti

#### Request data

Input browser tidak dipercaya. Validasi format, panjang, tipe, referensi resource, dan authorization.

#### PDO

Prepared statement memisahkan SQL structure dari data dan membantu mencegah SQL injection. Tetap validasi business rule.

#### Password

Gunakan password\_hash/password\_verify. Jangan menyimpan plaintext atau membuat algoritma hash sendiri.

#### Post/Redirect/Get

Setelah POST sukses, redirect mencegah resubmit saat refresh dan menghasilkan URL yang lebih jelas.

#### Upload

Batasi ukuran/type, generate nama acak, simpan sensitif di non-public, dan otorisasi saat download.

### Contoh kode lengkap

#### PHP

```
<?php
$dbpdo = new PDO($dsn, $dbUser, $dbPass, [
    PDO::ATTR_ERRMODE => PDO::ERRMODE_EXCEPTION,
    PDO::ATTR_DEFAULT_FETCH_MODE => PDO::FETCH_ASSOC,
]);

$email = filter_input(INPUT_POST, 'email', FILTER_VALIDATE_EMAIL);
$password = $_POST['password'] ?? '';

if (!$email || $password === '') {
    http_response_code(422);
    exit('Input tidak valid');
}

$stmt = $dbpdo->prepare('SELECT id, password_hash FROM users WHERE email = :email LIMIT 1');
$stmt->execute(['email' => $email]);
$user = $stmt->fetch();

if (!$user || !password_verify($password, $user['password_hash'])) {
    http_response_code(401);
```

```
        exit('Email atau password salah');  
    }  
  
    session_regenerate_id(true);  
    $_SESSION['user_id'] = $user['id'];  
    header('Location: /dashboard');  
    exit;
```

## Latihan praktik

1. Bangun mini blog CRUD dengan PDO.
2. Tambah login session dan logout.
3. Tambah CSRF token sederhana untuk form state-changing.
4. Tambah upload gambar dengan allowlist dan random filename.

### Checkpoint kompetensi

Bisa menjelaskan bagaimana login/session/validation/query/redirect bekerja tanpa Laravel dan dapat membangun mini CRUD aman secara dasar.



# 14. Composer, dependency, namespace, autoload

Composer mengelola dependency PHP dan autoloading. Memahami lockfile dan semantic versioning mencegah lingkungan development dan production berbeda secara tidak sengaja.

## Tujuan belajar

- ☐ Membaca composer.json/composer.lock.
- ☐ Membedakan install dan update.
- ☐ Memahami require vs require-dev.
- ☐ Menggunakan namespace dan PSR-4.

## Materi inti

### Lockfile

composer.lock merekam versi dependency yang terpilih. Untuk aplikasi, commit lockfile agar install konsisten.

### Install vs update

install mengikuti lockfile; update menyelesaikan constraint dan dapat mengubah lockfile. Update perlu review, test, dan security awareness.

### Autoload

PSR-4 memetakan namespace ke folder, sehingga require manual untuk tiap class tidak diperlukan.

## Contoh kode lengkap

JSON

```
{
  "require": {
    "php": "^8.5"
  },
  "autoload": {
    "psr-4": {
      "App\\": "src/"
    }
  },
  "require-dev": {
    "phpunit/phpunit": "^12"
  }
}
```

## Latihan praktik

1. Buat project kecil Composer dengan namespace App\.
2. Jalankan composer dump-autoload setelah menambah class.
3. Periksa composer outdated pada project latihan dan baca changelog sebelum update.

### Checkpoint kompetensi

Bisa menjelaskan mengapa composer.lock penting dan perbedaan dependency runtime vs development.

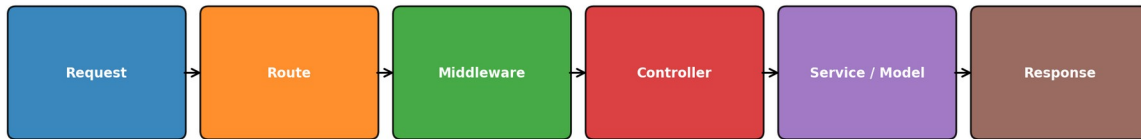


# 15. Laravel 13 - fondasi sampai fitur backend utama

Laravel dipelajari setelah HTTP, PHP, SQL, dan OOP dasar. Framework mempercepat kerja dengan konvensi, container, ORM, validation, auth, queue, cache, storage, dan testing.

## Laravel Request Lifecycle

Tambahkan lapisan hanya ketika kompleksitas benar-benar membutuhkannya.



## Tujuan belajar

- ☐ Memahami request lifecycle dan struktur project.
- ☐ Menguasai routing, controller, Blade, migration, Eloquent, validation.
- ☐ Menerapkan authentication, authorization, storage, logging.
- ☐ Menggunakan queue/cache/scheduler/API/testing sesuai kebutuhan.

## Materi inti

### Project structure

routes untuk pemetaan endpoint; app untuk kode aplikasi; resources untuk view/assets; database untuk migration/factory; config untuk konfigurasi; storage untuk file/log runtime.

### Routing/controller

Route harus memetakan request. Controller mengorkestrasi request-response, bukan menjadi tempat seluruh business logic.

### Eloquent

Model merepresentasikan data dan relationship. Gunakan eager loading untuk mencegah N+1 dan transaction untuk flow multi-step.

### Validation

Form Request cocok untuk validation kompleks dan authorization tertentu. Jangan percaya validation frontend saja.

### Authz

Authentication menjawab siapa; authorization menjawab boleh melakukan apa. Gunakan policy/gate, least privilege, dan test denial path.

## Infrastructure

Storage membedakan public/private, queue memindahkan kerja berat dari request, cache mempercepat read, scheduler menjalankan task terjadwal, logging membantu observability.

## API

Gunakan JSON Resource, pagination, error format konsisten, rate limit, auth strategy, dan idempotency bila operasi rentan duplicate.

## Testing

Feature test biasanya memberi nilai tinggi pada Laravel karena memverifikasi routing, middleware, validation, authz, database, dan response.

## Contoh kode lengkap

### PHP

```
// routes/web.php
Route::middleware('auth')->group(function () {
    Route::resource('posts', PostController::class);
});

// app/Http/Controllers/PostController.php
public function store(StorePostRequest $request): RedirectResponse
{
    $post = $request->user()->posts()->create($request->validated());
    return redirect()->route('posts.show', $post);
}

// app/Http/Requests/StorePostRequest.php
public function rules(): array
{
    return [
        'title' => ['required', 'string', 'max:200'],
        'body' => ['required', 'string', 'min:20'],
    ];
}
```

## Kesalahan umum

- Fat controller dan query langsung di Blade.
- Menampilkan tombol tersembunyi tetapi lupa authorization server.
- Menggunakan fill(\$request->all()) tanpa desain mass assignment.
- Menaruh kerja lambat seperti email/file processing di request synchronous.

## Latihan praktik

1. Buat modul berita Laravel lengkap: migration, model, controller, validation, Blade, policy, tests.
2. Tambahkan upload thumbnail public dan dokumen private.
3. Tambahkan job notification yang di-queue.
4. Tambahkan API Resource dan rate limit untuk endpoint public.

### Checkpoint kompetensi

Bisa membangun CRUD terproteksi dengan relationship, validation, policy, upload, test, dan menjelaskan request lifecycle Laravel.



# 16. Arsitektur Laravel yang sehat

Arsitektur yang baik mengikuti kompleksitas proyek. Aplikasi kecil tidak perlu repository/service/interface di setiap tempat. Tambahkan lapisan saat ada business flow yang benar-benar perlu dipisah atau diuji.

## Tujuan belajar

- ☐ Memisahkan HTTP, business logic, data, authorization, presentation.
- ☐ Mengenali kapan service/use-case diperlukan.
- ☐ Menghindari overengineering.
- ☐ Mendesain boundary untuk integration external.

## Mental model

Mulai sederhana. Tambahkan abstraction ketika ada duplikasi makna, boundary eksternal, complexity, atau alasan testing - bukan karena tutorial menyuruh.

### Inti konsep

Mulai sederhana. Tambahkan abstraction ketika ada duplikasi makna, boundary eksternal, complexity, atau alasan testing - bukan karena tutorial menyuruh.

## Materi inti

### Route

Pemetaan URL/method ke handler; jangan simpan business logic besar.

### Middleware

Cross-cutting request checks seperti auth/rate-limit. Jangan jadikan tempat query domain kompleks.

### Controller

Terjemahkan request -> use case -> response.

### Form Request

Validation/normalization input dan authorization tertentu.

### Service/Use Case

Workflow bisnis multi-step yang digunakan ulang atau perlu unit test tanpa HTTP.

### Model

Data, relationship, query scope, domain behavior ringan. Hindari god object.

### Policy

Authorization berbasis resource/action.

### Integration adapter

Membungkus API eksternal sehingga timeout, retry, mapping error, dan test dapat dikelola.

## Contoh kode lengkap

PHP

```
final class SubmitServiceRequest
{
    public function __construct(
        private TicketGenerator $tickets,
        private ServiceRequestRepository $requests,
    ) {}

    public function handle(User $user, array $data): ServiceRequest
    {
        return DB::transaction(function () use ($user, $data) {
            return $this->requests->create([
                ...$data,
                'user_id' => $user->id,
                'ticket' => $this->tickets->next(),
                'status' => 'pending',
            ]);
        });
    }
}
```

## Latihan praktik

1. Ambil controller 150+ baris (atau buat contoh) dan pisahkan berdasarkan tanggung jawab.
2. Buat adapter untuk satu external API dummy agar bisa diganti fake pada test.
3. Tulis ADR singkat: mengapa memilih monolith Laravel untuk proyek utama.

### Checkpoint kompetensi

Bisa menjelaskan struktur project berdasarkan tanggung jawab dan memilih kapan tidak perlu menambah layer.

# BAGIAN IV

Frontend modern: Node.js tooling, Vite, TypeScript, Vue 3, Inertia

## Arsitektur Full-Stack Modern

Authorization tetap di server; frontend tidak boleh menjadi sumber kebenaran izin.





## 17. Node.js, npm, dan Vite untuk tooling

Pada jalur ini Node.js terutama dipakai untuk toolchain frontend. Gunakan rilis LTS yang masih didukung untuk proyek stabil, dan pisahkan secret backend dari variable yang masuk bundle browser.

### Tujuan belajar

- ☐ Memahami package.json dan lockfile.
- ☐ Menjalankan npm install/dev/build.
- ☐ Memahami Vite dev server dan build output.
- ☐ Mengelola environment frontend dengan aman.

### Materi inti

#### Dependencies

dependencies diperlukan aplikasi saat runtime/build sesuai ekosistem; devDependencies untuk tooling development. Baca dokumentasi package karena konteks bundler dapat berbeda.

#### Lockfile

Commit package-lock.json agar dependency tree reproducible.

#### Vite

Development server cepat dan build production teroptimasi. Laravel mengintegrasikan Vite untuk asset pipeline modern.

#### Frontend env

Apa pun yang dibundle ke browser dapat dibaca user. Jangan pernah menaruh secret server di variable publik frontend.

### Contoh kode lengkap

#### BASH

```
npm install
npm run dev
npm run build

# package.json (potongan)
{
  "scripts": {
    "dev": "vite",
    "build": "vite build"
  }
}
```

### Latihan praktik

1. Buat project Vite kecil dan amati source vs dist.
2. Periksa package-lock setelah npm install.
3. Cari variable frontend yang seharusnya bukan secret.

#### Checkpoint kompetensi

Bisa menjelaskan perbedaan runtime browser, Node tooling, dan backend PHP; serta proses source -> build -> assets.

# 18. TypeScript

TypeScript menambahkan static type checking di atas JavaScript. Ia membantu menemukan mismatch sebelum runtime, tetapi tidak memvalidasi data eksternal secara otomatis.

## Tujuan belajar

- ☐ Menguasai primitive/object/array/union/literal/type/interface.
- ☐ Menggunakan narrowing, generic dasar, utility types.
- ☐ Mengaktifkan strict mode.
- ☐ Memahami perbedaan compile-time dan runtime validation.

## Materi inti

### Types

Gunakan type untuk memodelkan kontrak data. Union dan literal type sering lebih informatif daripada string bebas.

### Narrowing

TypeScript mempersempit tipe berdasarkan condition, typeof, in, discriminant, dan guard.

### Generics

Generic menghubungkan tipe input-output tanpa kehilangan informasi. Gunakan saat ada pola benar-benar generik.

### Runtime boundary

Response API tetap perlu validation/schema jika data tidak dipercaya; type annotation saja tidak mengubah payload runtime.

## Contoh kode lengkap

### TYPESCRIPT

```
type Role = 'superadmin' | 'admin' | 'operator' | 'user';

type User = {
  id: number;
  name: string;
  role: Role;
};

function canManageContent(user: User): boolean {
  return user.role === 'superadmin' || user.role === 'admin';
}

function assertNever(value: never): never {
  throw new Error(`Unhandled value: ${String(value)}`);
}
```

## Kesalahan umum

- Menggunakan any untuk menghilangkan semua error.
- Menganggap type frontend sebagai security boundary.

- Membuat generic kompleks padahal tipe concrete lebih jelas.

## Latihan praktik

1. Konversi satu file JS ke TS strict.
2. Modelkan state loading/success/error dengan discriminated union.
3. Buat generic Result<T,E> sederhana dan gunakan untuk function parsing.

### Checkpoint kompetensi

Bisa membaca error compiler, melakukan narrowing, dan menjelaskan mengapa TypeScript tidak menggantikan runtime validation.

# 19. Vue 3

Vue membangun UI berbasis component dan reactivity di atas HTML, CSS, JavaScript. Mulai dari local state dan component kecil sebelum menambah global store.

## Tujuan belajar

- ☐ Menguasai SFC, script setup, ref/reactive/computed.
- ☐ Menggunakan props, emits, slots, lifecycle, composable.
- ☐ Mendesain loading/error/empty state.
- ☐ Menjaga accessibility component.

## Materi inti

### Reactivity

ref cocok untuk nilai tunggal/primitive dan object; reactive untuk object proxy. computed untuk derived state, watch untuk side effect yang memang perlu.

### Props/emits

Props turun, event naik adalah model sederhana untuk komunikasi parent-child.

### Composable

Composable mengekstrak logic stateful reusable. Jangan memindahkan logic hanya untuk membuat file lebih banyak.

### Lifecycle

Gunakan onMounted untuk side effect yang membutuhkan DOM/browser; bersihkan listener/timer saat unmount jika dibuat manual.

## Contoh kode lengkap

### VUE

```
<script setup lang="ts">
import { computed, ref } from 'vue'

type Post = { id: number; title: string }
const query = ref('')
const posts = ref<Post[]>([
  { id: 1, title: 'Layanan Administrasi' },
  { id: 2, title: 'Agenda Gampong' },
])

const filtered = computed(() => {
  const q = query.value.trim().toLowerCase()
  return q ? posts.value.filter(p => p.title.toLowerCase().includes(q)) : posts.value
})
</script>

<template>
<label>
  Cari berita
  <input v-model="query" type="search">
</label>
<p v-if="filtered.length === 0">Tidak ada hasil.</p>
<ul v-else>
```

```
<li v-for="post in filtered" :key="post.id">{{ post.title }}</li>
</ul>
</template>
```

## Latihan praktik

1. Buat searchable list dengan computed.
2. Buat reusable modal yang bisa ditutup keyboard dan mengelola focus.
3. Buat form dengan validation error dari server.
4. Ekstrak data fetching ke composable jika dipakai ulang.

### Checkpoint kompetensi

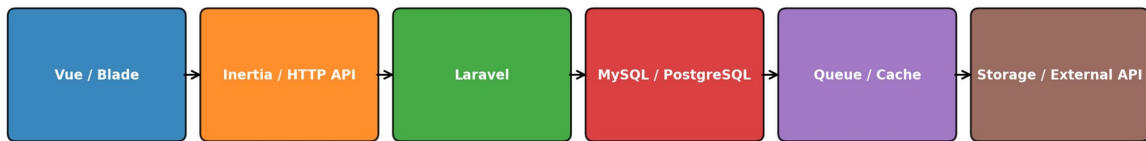
Bisa membangun mini app Vue tanpa library state global dan menjelaskan alur props/emits/reactivity.

## 20. Laravel + Vue + Inertia

Inertia memungkinkan routing/controller Laravel mengirim page props ke Vue tanpa memisahkan frontend menjadi SPA API deployment sejak awal. Authorization tetap berada di server.

### Arsitektur Full-Stack Modern

Authorization tetap di server; frontend tidak boleh menjadi sumber kebenaran izin.



### Tujuan belajar

- ☐ Memahami page component, shared props, form flow.
- ☐ Menangani validation error dari Laravel.
- ☐ Menggunakan layout dan navigation.
- ☐ Menentukan kapan Inertia vs REST SPA.

### Materi inti

#### Server-driven routing

Laravel tetap menentukan route/controller/policy; Vue menangani UI page.

#### Props

Kirim data minimum yang diperlukan. Jangan mengirim atribut sensitif hanya karena mudah.

#### Forms

Gunakan server validation sebagai sumber kebenaran, frontend untuk feedback cepat.

#### Architecture choice

Inertia cocok untuk monolith full-stack. REST SPA cocok bila frontend/backend benar-benar membutuhkan deployment/consumer terpisah.

### Contoh kode lengkap

#### PHP + VUE

```
// Controller
return Inertia::render('Posts/Index', [
    'posts' => PostResource::collection(
        Post::query()->latest()->paginate(15)
    ),
]);
```

```
<!-- Posts/Index.vue -->
<script setup lang="ts">
defineProps<{ posts: { data: Array<{ id:number; title:string }> } }>()
</script>

<template>
  <article v-for="post in posts.data" :key="post.id">
    <h2>{{ post.title }}</h2>
  </article>
</template>
```

## Latihan praktik

1. Migrasikan satu Blade list menjadi Inertia/Vue tanpa mengubah backend authorization.
2. Tambahkan form create dengan server validation.
3. Pastikan props tidak mengandung field private yang tidak dibutuhkan.

### Checkpoint kompetensi

Bisa membangun satu vertical slice Laravel + Inertia + Vue dari route sampai database dan tahu di mana authorization dilakukan.



## 21. State management dan data fetching

Global state bukan default. Pisahkan UI state lokal, shared client state, dan server state. Setiap cache atau optimistic update harus memiliki aturan invalidation dan recovery.

### Tujuan belajar

- ☐ Memilih local state vs lifted state vs store.
- ☐ Menggunakan composable untuk reusable logic.
- ☐ Membedakan server state dan UI state.
- ☐ Memahami caching/invalidation/optimistic update.

### Materi inti

#### Local first

Simpan state sedekat mungkin dengan component yang menggunakannya.

#### Pinia

Gunakan store global saat state benar-benar lintas halaman/component dan lifecycle-nya jelas.

#### Server state

Data dari server dapat stale, gagal, atau berubah di tempat lain; cache perlu refetch/invalidation policy.

#### Optimistic UI

Update UI sebelum server sukses hanya aman jika rollback/reconciliation saat gagal sudah didesain.

### Latihan praktik

1. Identifikasi state dari dashboard dan klasifikasikan local/shared/server.
2. Buat composable useSearchQuery.
3. Implementasikan optimistic toggle dengan rollback pada failure di project latihan.

#### Checkpoint kompetensi

Bisa menjelaskan mengapa tidak semua state harus masuk store global dan bagaimana data server menjadi stale.

# BAGIAN V

Quality engineering: testing, security, performance, observability

## Strategi Pengujian

Semakin ke kanan semakin mahal; lindungi alur paling kritis dengan kombinasi test yang tepat.



## 22. Testing strategy

Testing memberi kepercayaan saat kode berubah. Targetnya bukan coverage 100%, tetapi perlindungan terhadap aturan penting, jalur kritis, boundary, dan bug yang pernah terjadi.

### Strategi Pengujian

Semakin ke kanan semakin mahal; lindungi alur paling kritis dengan kombinasi test yang tepat.



### Tujuan belajar

- ☐ Membedakan unit, feature, integration, E2E, smoke.
- ☐ Menulis happy path dan failure path.
- ☐ Membuat deterministic tests.
- ☐ Menggunakan regression test setelah bug.

### Materi inti

#### Unit

Cepat, fokus pada pure business logic.

#### Feature

Pada Laravel, sangat bernilai untuk memverifikasi request -> middleware -> validation -> DB -> response.

#### Integration

Mengujikan boundary antar komponen, database, filesystem, atau wrapper API.

#### E2E

Sedikit tetapi penting untuk critical user journey seperti login -> submit -> cek status.

#### Test data

Factory membuat data yang valid. Hindari ketergantungan pada urutan test atau waktu nyata tanpa kontrol.

### Contoh kode lengkap

#### PHPUNIT / LARAVEL

```

public function test_user_cannot_update_other_users_post(): void
{
    $owner = User::factory()->create();
    $other = User::factory()->create();
  
```

```
$post = Post::factory()->for($owner)->create();

$this->actingAs($other)
    ->put(route('posts.update', $post), [
        'title' => 'Tidak boleh',
        'body' => 'Isi yang valid untuk request',
    ])
    ->assertForbidden();
}
```

## Latihan praktik

1. Tulis test untuk validation failure.
2. Tulis authorization denial test.
3. Reproduksi satu bug lalu buat regression test sebelum fix.
4. Buat smoke checklist setelah deploy.

### Checkpoint kompetensi

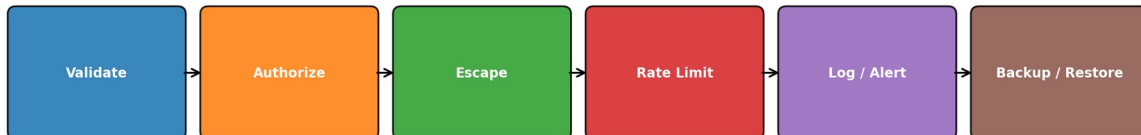
Test suite melindungi happy path, validation, authorization, dan satu critical flow tanpa bergantung pada data manual.

## 23. Web security wajib dan OWASP mindset

Keamanan adalah proses desain, implementasi, konfigurasi, update, monitoring, dan recovery. OWASP Top 10:2025 menekankan broken access control, misconfiguration, software supply chain, cryptographic failures, injection, insecure design, authentication, integrity, logging/alerting, dan exceptional conditions.

### Defense in Depth

Keamanan bukan satu fitur; setiap lapisan mengurangi dampak kegagalan lapisan lain.



### Tujuan belajar

- ☐ Menerapkan validation, escaping, prepared query, CSRF protection, authz.
- ☐ Mengamankan session/cookie/upload/secret.
- ☐ Memahami supply chain dan dependency risk.
- ☐ Membuat threat model sederhana.

### Materi inti

#### Threat modeling

Identifikasi asset, actor, entry point, trust boundary, abuse case, dan control. Prioritaskan berdasarkan dampak dan kemungkinan.

#### Broken access control

Setiap resource access harus diverifikasi server-side. Jangan percaya ID dari URL atau role dari browser.

#### Injection/XSS

Parameterize query, escape output sesuai context, hindari raw HTML yang tidak perlu.

#### Auth/session

Hash password, regenerate session setelah login, gunakan secure cookie, rate limit, dan recovery flow yang aman.

#### Upload

Size/type allowlist, random filename, non-public storage untuk sensitif, jangan eksekusi upload.

#### Supply chain

Lock dependency, update dengan review, cek advisory, minimalkan package, lindungi CI secret.

## Logging

Log event keamanan yang berguna tanpa password/token/NIK mentah. Alert harus actionable.

## Exceptional conditions

Error handling yang buruk dapat membuka data atau membuat aplikasi fail-open. Default ke kondisi aman.

## Latihan praktik

1. Buat threat model login, upload, dan workflow admin.
2. Audit semua endpoint: auth? authz? validation? rate limit? log?
3. Buat test IDOR: user A mencoba resource user B.
4. Periksa dependency dan hapus package yang tidak digunakan.

### Checkpoint kompetensi

Bisa menemukan minimal satu risiko pada input, access control, session, upload, dependency, logging, dan recovery; serta menunjukkan control server-side.

## 24. Performance engineering

Optimasi harus dimulai dari pengukuran. Bottleneck paling sering berada pada network, query/database, image/assets, pekerjaan synchronous berat, atau kode yang mengulang pekerjaan.

### Tujuan belajar

- ☐ Mengukur sebelum optimasi.
- ☐ Membaca network waterfall dan slow query.
- ☐ Menggunakan pagination, index, eager loading, cache, queue.
- ☐ Memahami load test/profiling dasar.

### Materi inti

#### Frontend

Kurangi JS yang tidak perlu, optimalkan image, gunakan caching, code splitting sesuai kebutuhan, hindari layout shift.

#### Backend

Periksa query count/latency, N+1, serialization besar, external API timeout, pekerjaan CPU/I/O yang bisa di-queue.

#### Cache

Cache data yang mahal dibaca dan punya aturan invalidation jelas. Cache bug adalah bug konsistensi.

#### Load

Load test pada environment aman/staging dengan skenario realistis; jangan hanya mengejar requests/sec tanpa latency/error rate.

### Latihan praktik

1. Ukur halaman sebelum/after optimasi gambar.
2. Temukan N+1 lalu perbaiki dengan eager loading.
3. Tambahkan pagination pada daftar besar.
4. Pindahkan email berat ke queue dan bandingkan response time.

#### Checkpoint kompetensi

Punya baseline metric, perubahan optimasi terukur, dan dapat menjelaskan trade-off cache/index/queue.

## 25. Observability dan debugging production

Production debugging membutuhkan log, metrics, error tracking, health check, dan runbook. Tanpa observability, tim hanya menerima laporan "website error" tanpa konteks.

### Tujuan belajar

- ☐ Mendesain structured logging.
- ☐ Memantau latency, error rate, queue, database.
- ☐ Membuat health check dan alert.
- ☐ Menulis runbook incident sederhana.

### Materi inti

#### Logs

Gunakan level dan context: request ID, user ID non-sensitif, route, status. Jangan log secret.

#### Metrics

Golden signals sederhana: traffic, latency, error, saturation. Untuk job: queue depth, age, failure.

#### Health

Liveness memastikan process hidup; readiness memastikan app siap melayani dependency penting. Jangan membuat health check yang terlalu berat.

#### Runbook

Dokumen langkah aman: verifikasi dampak, cek dashboard/log, mitigasi, rollback, komunikasi, postmortem.

### Latihan praktik

1. Tambahkan correlation ID ke log request.
2. Buat endpoint health minimal.
3. Tulis runbook "queue worker berhenti".
4. Simulasikan error dan pastikan log cukup untuk diagnosis tanpa data sensitif.

#### Checkpoint kompetensi

Bisa menjawab: apa yang rusak, sejak kapan, seberapa luas, error apa, dependency mana, dan apa langkah mitigasi.



# BAGIAN VI

Production: Linux, network, deployment, Docker, CI/CD, backup

## Pipeline Production

Deploy aman selalu punya backup, observability, dan rollback plan.



## 26. Linux dan server dasar

Sebagian besar server production berbasis Linux. Kamu tidak perlu menjadi sysadmin penuh, tetapi harus mampu menavigasi filesystem, process, permission, network, service, log, dan resource usage.

### Tujuan belajar

- ☐ Menggunakan command dasar Linux dengan aman.
- ☐ Memahami user/group/permission/process/service.
- ☐ Memeriksa port, disk, memory, log.
- ☐ Menggunakan SSH dan environment variable secara aman.

### Materi inti

#### Permissions

Least privilege berlaku juga pada filesystem. Web server tidak perlu permission lebih dari yang dibutuhkan.

#### Processes/services

systemd mengelola service pada banyak distro. Ketahui cara status/start/restart dan membaca journal/log.

#### Network

ss/lsof membantu melihat port listening. Firewall membatasi traffic, tetapi aplikasi tetap perlu authz.

#### Disk

Storage/log/upload dapat memenuhi disk dan menjatuhkan service; monitor kapasitas dan rotasi log.

### Contoh kode lengkap

#### BASH

```
pwd
ls -la
cd /var/www/app
ps aux
ss -tulpn
df -h
free -h
tail -f storage/logs/laravel.log

# contoh service
systemctl status nginx
systemctl status php-fpm
```

### Latihan praktik

1. Gunakan VM/container lokal untuk latihan command tanpa menyentuh server penting.
2. Cari process dan port server lokal.
3. Buat user non-root latihan dan uji permission file.

#### Checkpoint kompetensi

Bisa masuk server latihan, menemukan app/log/process/port, serta melakukan diagnosis dasar tanpa mengandalkan GUI.



## 27. DNS, domain, HTTPS, web server

Saat user membuka domain, DNS memetakan nama ke alamat, TLS mengamankan koneksi, web server/reverse proxy menerima request, lalu meneruskannya ke runtime aplikasi.

### Tujuan belajar

- ☐ Memahami A/AAAA/CNAME secara konseptual.
- ☐ Memahami TLS certificate dan HTTPS redirect.
- ☐ Memahami Nginx/Apache virtual host dan PHP-FPM.
- ☐ Menyelaraskan upload/body/timeouts dengan aplikasi.

### Materi inti

#### DNS

Perubahan DNS memiliki propagation/cache. TTL menentukan berapa lama resolver menyimpan record.

#### TLS

Certificate membuktikan identitas domain dan memungkinkan enkripsi transport. Gunakan automation renew.

#### Reverse proxy

Web server dapat terminate TLS, serve static assets, enforce limits, dan meneruskan dynamic request ke app runtime.

#### Headers

Atur security/caching headers dengan pemahaman; jangan copy konfigurasi acak tanpa tahu efek.

### Latihan praktik

1. Gambar alur domain -> DNS -> server -> Nginx -> PHP-FPM -> Laravel.
2. Periksa headers response pada staging.
3. Uji upload limit di proxy dan app agar konsisten.

#### Checkpoint kompetensi

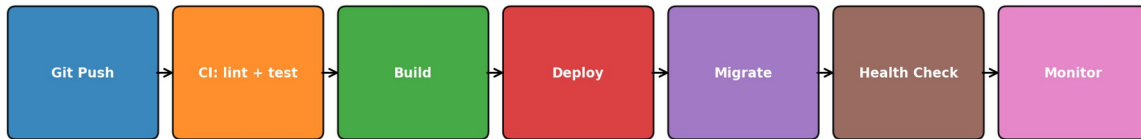
Bisa menjelaskan request dari domain hingga Laravel dan titik-titik yang dapat menyebabkan 404/413/502/timeout.

## 28. Deployment Laravel

Deploy adalah perubahan production yang harus dapat diverifikasi dan dibalik. Jangan mengandalkan "upload file lalu semoga jalan".

### Pipeline Production

Deploy aman selalu punya backup, observability, dan rollback plan.



### Tujuan belajar

- ☐ Menyiapkan build/test sebelum deploy.
- ☐ Mengelola env, secret, migration, queue, scheduler.
- ☐ Melakukan health check dan smoke test.
- ☐ Memiliki backup dan rollback plan.

### Materi inti

#### Pre-deploy

CI hijau, dependency lock, build assets, config valid, backup jika perubahan berisiko.

#### Migration

Migration production harus mempertimbangkan data besar, lock, backward compatibility, dan rollback realistis.

#### Workers

Queue worker perlu direstart/reload sesuai release supaya kode baru digunakan.

#### Post-deploy

Health check, critical smoke test, error rate, logs. Rollback lebih cepat daripada debugging panjang saat incident kritis.

### Contoh kode lengkap

#### BASH

```
# gambaran, sesuaikan platform
composer install --no-dev --prefer-dist --optimize-autoloader
npm ci
npm run build
php artisan config:cache
```

```
php artisan route:cache
php artisan migrate --force
php artisan queue:restart
php artisan about
```

## Latihan praktik

1. Tulis checklist deploy staging.
2. Buat rencana rollback migration + code.
3. Lakukan deployment staging dari Git dan dokumentasikan semua command.

### Checkpoint kompetensi

Bisa deploy project latihan ke staging dengan HTTPS, env benar, migration, queue, logs, health check, dan langkah rollback terdokumentasi.

## 29. Docker

Docker memaketkan runtime dan dependency, bukan menggantikan pemahaman aplikasi, database, Linux, atau network. Pelajari setelah aplikasi bisa berjalan tanpa Docker.

### Tujuan belajar

- ☐ Memahami image, container, layer, port, volume, network.
- ☐ Menulis Dockerfile sederhana.
- ☐ Menggunakan docker compose untuk environment lokal.
- ☐ Menghindari secret baked into image.

### Materi inti

#### Image/container

Image immutable template; container instance runtime.

#### Layer/cache

Urutan Dockerfile memengaruhi build cache. Copy lockfile/install dependency sebelum source bila sesuai untuk cache lebih baik.

#### Volume

Data persistent seperti database tidak boleh hilang bersama container.

#### Network

Service dalam compose berkomunikasi lewat network internal dan service name, bukan localhost host machine.

### Contoh kode lengkap

#### DOCKER

```
FROM php:8.5-cli AS app
WORKDIR /app
COPY . /app
CMD ["php", "artisan", "serve", "--host=0.0.0.0", "--port=8000"]

# docker-compose.yml (konsep ringkas)
services:
  app:
    build: .
    ports: ["8000:8000"]
  db:
    image: mysql:8
    volumes: ["db-data:/var/lib/mysql"]
volumes:
  db-data: {}
```

### Latihan praktik

1. Containerize app latihan lokal.
2. Matikan/recreate container database dan pastikan volume menjaga data.
3. Cek ukuran image dan identifikasi layer besar.

**Checkpoint kompetensi**

Bisa menjelaskan image/container/volume/network dan menjalankan stack lokal tanpa menyimpan secret di Dockerfile.



## 30. CI/CD

CI menguji perubahan secara otomatis; CD menyiapkan atau menjalankan delivery/deployment. Pipeline adalah gerbang kualitas, bukan sekadar badge.

### Tujuan belajar

- ☐ Membuat pipeline install -> lint/static analysis -> test -> build.
- ☐ Memisahkan secret CI dari repository.
- ☐ Menambahkan protected branch/review.
- ☐ Melakukan smoke test dan rollback saat deploy gagal.

### Materi inti

#### CI

Setiap push/PR menjalankan pemeriksaan konsisten di environment bersih.

#### Artifacts

Build output dapat disimpan sebagai artifact dan digunakan tahap deploy.

#### Secrets

Gunakan secret management platform CI. Scope minimal, rotate bila bocor.

#### Deployment

Gunakan environment protection dan approval untuk production bila sesuai risiko.

### Contoh kode lengkap

#### YAML

```
name: test
on: [push, pull_request]
jobs:
  app:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: shivammathur/setup-php@v2
        with: { php-version: '8.5' }
      - run: composer install --no-interaction --prefer-dist
      - run: php artisan test
      - uses: actions/setup-node@v4
        with: { node-version: '24', cache: 'npm' }
      - run: npm ci
      - run: npm run build
```

### Latihan praktik

1. Buat CI untuk project Laravel latihan.
2. Sengaja buat test gagal dan amati pipeline.
3. Tambahkan build frontend dan artifact.
4. Buat checklist secret dan permission token CI.

**Checkpoint kompetensi**

Pull request tidak bisa dianggap siap jika test/build gagal, dan pipeline tidak membutuhkan secret yang tersimpan di repo.

## 31. Backup dan disaster recovery

Backup yang tidak pernah diuji restore belum terbukti berguna. Recovery harus mempertimbangkan database, uploaded files, config, secret, dan prosedur operasional.

### Tujuan belajar

- ☐ Membuat strategi backup database + storage.
- ☐ Memahami retention, encryption, off-site copy.
- ☐ Memahami RPO/RTO secara konseptual.
- ☐ Melakukan restore test.

### Materi inti

#### RPO

Berapa banyak data terbaru yang dapat hilang secara maksimum.

#### RTO

Berapa lama layanan boleh tidak tersedia.

#### 3-2-1 mindset

Simpan beberapa salinan di media/lokasi terpisah sesuai risiko. Jangan mengandalkan satu disk/server.

#### Restore drill

Jadwalkan uji restore ke environment terisolasi dan dokumentasikan waktu/langkah.

### Latihan praktik

1. Tulis kebijakan backup untuk portal: frekuensi DB, file, retention.
2. Lakukan backup dan restore database latihan.
3. Buat runbook recovery server hilang.

#### Checkpoint kompetensi

Bisa memulihkan project latihan dari backup ke environment baru dengan langkah terdokumentasi.

# BAGIAN VII

Go untuk backend specialization

## Jalur Belajar Go

Concurrency dipelajari setelah fondasi bahasa dan backend stabil.



## 32. Fondasi Go

Go dipelajari setelah backend fundamental agar kamu mempelajari bahasa dan ekosistem baru, bukan sekaligus belajar HTTP, database, dan arsitektur dari nol.

### Tujuan belajar

- ☐ Menggunakan go toolchain/module/package.
- ☐ Menguasai variable, function, struct, method, pointer, slice, map, interface.
- ☐ Memahami explicit error handling dan defer.
- ☐ Menulis test dengan standard library.

### Materi inti

#### Design

Go sengaja kecil dan eksplisit. Idiom penting: composition, small interfaces, errors as values, formatting standar.

#### Pointers

Gunakan pointer ketika perlu mutate/share large value atau semantics identity. Jangan berpikir pointer sebagai "lebih cepat" secara otomatis.

#### Interfaces

Interface di Go dipenuhi secara implicit. Biasanya didefinisikan dekat consumer dan kecil.

#### Errors

Return error dan tambahkan context. Jangan menelan error. panic bukan pengganti error flow normal.

### Contoh kode lengkap

GO

```
package main

import (
    "fmt"
)

func total(price, qty int) (int, error) {
    if price < 0 || qty < 0 {
        return 0, fmt.Errorf("invalid value")
    }
    return price * qty, nil
}

func main() {
    value, err := total(15000, 2)
    if err != nil {
        panic(err)
    }
    fmt.Println(value)
}
```

### Latihan praktik

1. Buat CLI inventory Go dengan slice/map.

2. Buat package terpisah dan unit test.
3. Implementasikan interface kecil untuk storage in-memory.

#### **Checkpoint kompetensi**

Bisa membuat module Go kecil, menulis test, dan menjelaskan struct/interface/error tanpa membandingkan sintaks saja dengan PHP.

## 33. Concurrency Go

Concurrency kuat, tetapi tidak berarti setiap function menjadi goroutine. Mulai dari goroutine, channel, synchronization, context, timeout, dan race awareness setelah code synchronous sudah benar.

### Jalur Belajar Go

Concurrency dipelajari setelah fondasi bahasa dan backend stabil.



### Tujuan belajar

- ☐ Memahami goroutine/channel/select.
- ☐ Menggunakan WaitGroup/mutex ketika tepat.
- ☐ Menggunakan context cancellation/timeout.
- ☐ Menghindari race dan goroutine leak.

### Materi inti

#### Goroutine

Unit concurrent ringan yang dijadwalkan runtime Go.

#### Channel

Mekanisme komunikasi/synchronization. Tidak semua shared state perlu channel; mutex juga valid.

#### Context

Context membawa cancellation/deadline melintasi request boundary. Jangan menyimpan context di struct jangka panjang.

#### Race

Shared mutable state tanpa synchronization dapat race. Gunakan race detector dalam development/test.

### Contoh kode lengkap

GO

```

ctx, cancel := context.WithTimeout(context.Background(), 2*time.Second)
defer cancel()

ch := make(chan string, 1)
go func() {
    ch <- "selesai"
}
  
```

```

}()

select {
case result := <-ch:
    fmt.Println(result)
case <-ctx.Done():
    fmt.Println("timeout")
}

```

## Latihan praktik

1. Buat worker pool kecil untuk memproses daftar item.
2. Tambahkan timeout/cancellation.
3. Jalankan go test -race pada contoh shared counter.

### Checkpoint kompetensi

Bisa menjelaskan kapan concurrency bermanfaat, kapan mutex lebih sederhana, dan bagaimana mencegah goroutine terus hidup setelah request selesai.



## 34. Backend Go

Standard library Go cukup kuat untuk HTTP server, JSON, context, database/sql, testing, logging, dan graceful shutdown. Framework dapat ditambahkan setelah kebutuhan jelas.

### Tujuan belajar

- ☐ Membangun HTTP JSON API.
- ☐ Menggunakan database/sql + transaction + prepared/parameterized query.
- ☐ Membuat middleware dan config.
- ☐ Menerapkan graceful shutdown dan HTTP tests.

### Materi inti

#### HTTP

Handler menerima request dan menulis response. Pisahkan decode/validate/use-case/encode.

#### Database

database/sql adalah abstraction pool. Selalu bawa context, cek error, tutup rows, gunakan transaction untuk unit kerja.

#### Shutdown

Server production perlu berhenti dengan grace period agar request yang berjalan dapat selesai.

#### Testing

httptest memudahkan test handler tanpa membuka port nyata.

### Contoh kode lengkap

#### GO

```
func health(w http.ResponseWriter, r *http.Request) {
    w.Header().Set("Content-Type", "application/json")
    json.NewEncoder(w).Encode(map[string]string{"status": "ok"})
}

func main() {
    mux := http.NewServeMux()
    mux.HandleFunc("GET /health", health)

    srv := &http.Server{
        Addr:           ":8080",
        Handler:        mux,
        ReadHeaderTimeout: 5 * time.Second,
    }

    if err := srv.ListenAndServe(); err != nil && err != http.ErrServerClosed {
        log.Fatal(err)
    }
}
```

### Latihan praktik

1. Buat CRUD JSON kecil dengan in-memory storage lalu database.
2. Tambahkan request validation dan error format konsisten.

3. Tambahkan HTTP test untuk 200/400/404.
4. Tambahkan graceful shutdown.

### **Checkpoint kompetensi**

Bisa membangun dan menguji API Go kecil dengan struktur jelas dan error handling konsisten.

## 35. Laravel vs Go: memilih berdasarkan kebutuhan

Tidak ada kebutuhan memigrasikan Laravel ke Go hanya karena benchmark. Pilih berdasarkan requirement, produktivitas tim, latency, concurrency, operasi, dan bottleneck nyata.

### Tujuan belajar

- ☐ Membandingkan trade-off secara factual.
- ☐ Memilih stack untuk jenis workload.
- ☐ Menghindari premature microservices.

### Materi inti

#### Laravel

Sangat produktif untuk CRUD, CMS, admin, business app, auth, queue, dan ekosistem web terpadu.

#### Go

Sangat cocok untuk service jaringan concurrent, worker/CLI ringkas, binary deployment, dan workload yang membutuhkan kontrol resource eksplisit.

#### Monolith first

Monolith modular lebih mudah dioperasikan. Pecah service setelah boundary dan bottleneck terbukti, bukan demi tren.

### Contoh keputusan

- Portal gampong/CMS/admin: Laravel lebih cepat mencapai fitur lengkap.
- Worker pemrosesan concurrent dengan kebutuhan isolasi deployment: Go dapat dipertimbangkan.
- API internal sederhana: keduanya mampu; pilih berdasarkan tim dan operasi.
- Microservice: butuh alasan organisasi/skalabilitas/boundary yang jelas.

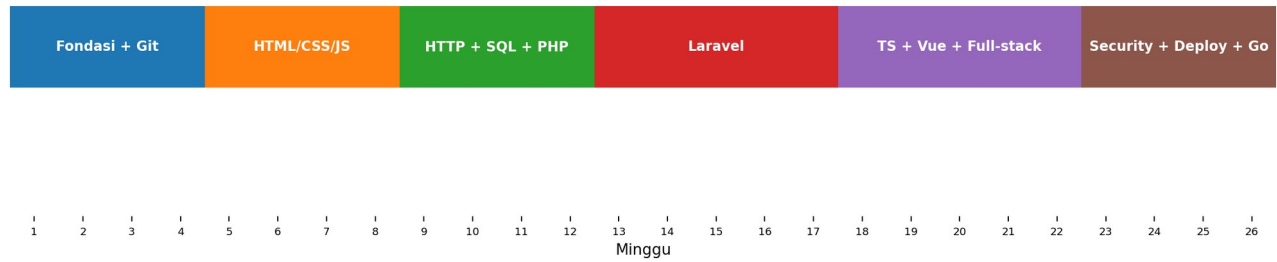
#### Checkpoint kompetensi

Bisa menjelaskan alasan memilih stack dengan requirement dan trade-off, bukan label "lebih cepat" atau "lebih modern".

# BAGIAN VIII

Proyek terpadu, roadmap eksekusi, debugging, refactoring, portfolio

## Roadmap 26 Minggu - Kecepatan Fleksibel



## 36. Project utama: Portal Gampong bertumbuh 15 level

Gunakan satu proyek nyata yang tumbuh seiring kemampuan. Setiap level menambah satu lapisan teknologi tanpa menghapus fondasi yang sudah dipahami.

### Tujuan belajar

- ☐ Membangun vertical slice bertahap.
- ☐ Menghubungkan teori dengan artefak nyata.
- ☐ Menyimpan seluruh perubahan di Git dan README.

### Materi inti

#### Level 1-3

HTML statis -> responsive CSS -> JavaScript UI.

#### Level 4-7

PHP rendering -> MySQL -> auth -> CRUD berita.

#### Level 8-12

Migrasi Laravel -> dashboard -> RBAC -> REST API -> Vue + TypeScript.

#### Level 13-15

Testing -> security hardening -> production deploy.

### Matriks 15 level

Setiap level memiliki definition of done: fitur berjalan, kasus gagal ditangani, code tersimpan di Git, README diperbarui, dan kamu bisa menjelaskan arsitekturnya.

### Latihan praktik

1. Buat board 15 level dengan issue per level.
2. Untuk setiap level, simpan screenshot/demo, test atau langkah verifikasi, dan refleksi keputusan.

#### Checkpoint kompetensi

Project bisa ditunjukkan sebagai cerita perkembangan kemampuan, bukan hanya final ZIP.

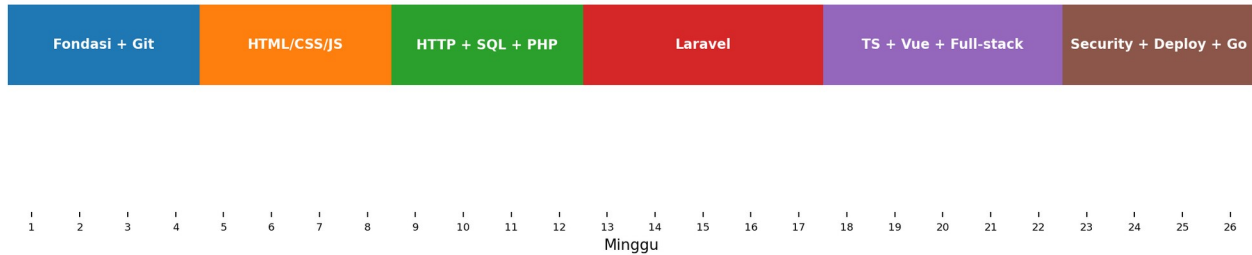
| Level | Target             | Bukti selesai                          |
|-------|--------------------|----------------------------------------|
| 1     | HTML statis        | Semantic layout + navigation           |
| 2     | Responsive CSS     | Mobile/desktop/zoom/dark mode          |
| 3     | JavaScript UI      | Navbar/search/form states              |
| 4     | PHP sederhana      | Server-rendered request/response       |
| 5     | MySQL              | Schema + CRUD + relationship           |
| 6     | Authentication     | Register/login/session/logout          |
| 7     | CRUD berita        | Create/edit/delete/list/detail         |
| 8     | Laravel            | Route/controller/Blade/Eloquent        |
| 9     | Admin dashboard    | CMS + validation + upload              |
| 10    | RBAC               | superadmin/admin/operator/user/guest   |
| 11    | REST API           | JSON resource + rate limit + errors    |
| 12    | Vue + TypeScript   | Component + state + form + Inertia/API |
| 13    | Testing            | Unit/feature/critical flow             |
| 14    | Security hardening | authz/XSS/CSRF/upload/secret           |
| 15    | Production         | HTTPS/backup/queue/logs/CI-CD          |



# 37. Roadmap 26 minggu

Roadmap adalah urutan belajar, bukan deadline kaku. Jika checkpoint belum tercapai, ulangi minggu tersebut daripada menumpuk kebingungan.

## Roadmap 26 Minggu - Kecepatan Fleksibel



## Tujuan belajar

- ☐ Memiliki target mingguan yang konkret.
- ☐ Mengevaluasi output, bukan hanya jam belajar.
- ☐ Menyesuaikan kecepatan tanpa mengubah urutan dependency konsep.

## Materi inti

### Minggu 1-4

Environment, Git, PHP, function/array/error, OOP/algoritma.

### Minggu 5-8

HTML, CSS, JavaScript, DOM/fetch/DevTools.

### Minggu 9-12

HTTP/JSON, SQL/schema, PHP web/PDO.

### Minggu 13-17

Laravel routing, Eloquent, authz, upload, queue/cache/API/testing.

### Minggu 18-22

Vite, TypeScript, Vue, Inertia/full-stack.

### Minggu 23-26

Security audit, Linux/deploy, Docker/CI-CD/backup, Go + portfolio.

### Checkpoint kompetensi

Setiap minggu menghasilkan artefak: latihan, commit, mini project, test, atau deployment - bukan hanya playlist yang selesai ditonton.

| Minggu | Fokus                          | Output                    |
|--------|--------------------------------|---------------------------|
| 1      | Environment, terminal, Git     | Repo belajar + CLI drills |
| 2      | PHP variables, condition, loop | 20 latihan logika         |

|    |                                  |                                 |
|----|----------------------------------|---------------------------------|
| 3  | Function, array, error           | CLI mini apps                   |
| 4  | OOP + algoritma dasar            | Mini inventory CLI              |
| 5  | HTML semantic                    | Landing page                    |
| 6  | CSS responsive                   | Responsive redesign             |
| 7  | JavaScript fundamental           | Interactive UI                  |
| 8  | DOM, fetch, DevTools             | Frontend mini project           |
| 9  | HTTP + JSON                      | Request lab                     |
| 10 | SQL CRUD                         | Database exercises              |
| 11 | JOIN + schema design             | Mini schema gampong             |
| 12 | PHP web + PDO                    | Mini blog                       |
| 13 | Laravel routing/controller/Blade | News app                        |
| 14 | Migration/Eloquent/relationship  | Data model app                  |
| 15 | Validation/auth/policy           | Protected CRUD                  |
| 16 | Upload/mail/logging              | Admin module                    |
| 17 | Queue/cache/API/testing          | Backend checkpoint              |
| 18 | Node/Vite + TypeScript           | TS exercises                    |
| 19 | Vue component/reactivity         | Vue mini app                    |
| 20 | Vue form/composable              | Dashboard UI                    |
| 21 | Laravel + Vue/Inertia            | Full-stack module               |
| 22 | API integration/testing          | Full-stack checkpoint           |
| 23 | Security audit                   | Fix security checklist          |
| 24 | Linux + deployment               | Staging deployment              |
| 25 | Docker + CI/CD + backup          | Automated pipeline              |
| 26 | Go fundamental + reflection      | Go API mini project + portfolio |



## 38. Rutinitas belajar mingguan

Konsistensi mengalahkan marathon sporadis. Gunakan siklus konsep -> latihan -> fitur -> debug/test -> refactor -> checkpoint -> review.

### Tujuan belajar

- ☐ Membagi belajar menjadi aktivitas berbeda.
- ☐ Menghindari tutorial binge tanpa praktik.
- ☐ Mencatat pertanyaan dan bug.

### Materi inti

#### Senin

Konsep baru dan mental model.

#### Selasa

5-10 latihan kecil.

#### Rabu

Implementasi fitur project.

#### Kamis

Debug + test kasus gagal.

#### Jumat

Refactor naming/structure.

#### Sabtu

Project checkpoint/vertical slice.

#### Minggu

Review ringan, README, rencana berikutnya.

### Latihan praktik

1. Buat log belajar satu halaman per minggu: apa dipelajari, bug, konsep lemah, target berikut.
2. Tentukan jam belajar realistis dan lindungi waktu praktik coding sendiri.

#### Checkpoint kompetensi

Setelah 4 minggu, terlihat bukti peningkatan dalam Git history dan kemampuan menjelaskan, bukan sekadar jumlah video yang ditonton.

## 39. Sistem checkpoint dan asesmen diri

Checkpoint mencegah ilusi kompetensi. Kamu naik level jika bisa menjelaskan, membangun, memodifikasi, debug, test, dan mendokumentasikan.

### Tujuan belajar

- ☐ Menggunakan rubric objektif.
- ☐ Mengenali gap sebelum lanjut.
- ☐ Membuktikan kompetensi dengan artefak.

### Materi inti

#### Jelaskan

Tanpa catatan, jelaskan mental model dan trade-off.

#### Bangun

Buat contoh dari nol.

#### Modifikasi

Ubah requirement tanpa mengulang tutorial.

#### Debug

Cari bug dari error/observability.

#### Uji

Tulis test atau verification checklist.

#### Dokumentasi

README setup dan keputusan.

#### Checkpoint kompetensi

Skor diri bukan angka prestise. Jika satu kemampuan penting masih "tidak bisa tanpa menyalin", ulangi topik dengan project lebih kecil.

## 40. Debugging framework

Debugging sistematis lebih cepat daripada mengubah banyak file. Tujuannya mengecilkan ruang kemungkinan sampai menemukan fakta yang bertentangan dengan harapan.

### Tujuan belajar

- ☐ Mereproduksi bug konsisten.
- ☐ Membaca stack trace penuh.
- ☐ Menentukan layer bug.
- ☐ Menguji satu hipotesis per perubahan.

### Mental model

Jangan mulai dari "bagian mana yang kelihatan mencurigakan". Mulai dari fakta: apa expected, apa actual, dan boundary pertama tempat keduanya berbeda.

#### Inti konsep

Jangan mulai dari "bagian mana yang kelihatan mencurigakan". Mulai dari fakta: apa expected, apa actual, dan boundary pertama tempat keduanya berbeda.

### Materi inti

#### Lapisan

UI -> network -> route -> validation -> authorization -> business logic -> query -> storage -> deployment.

#### Evidence

Catat input aktual, state, query, response, log, dan kondisi environment.

#### Regression

Setelah fix, tambahkan test atau langkah verifikasi agar bug tidak kembali.

### Latihan praktik

1. Debug bug 422 validation dengan Network + log.
2. Debug N+1 dengan query log/profiler.
3. Debug 403 dengan policy/role data.
4. Debug production-only config mismatch dengan environment checklist.

#### Checkpoint kompetensi

Bisa membuat bug report lengkap: steps, expected, actual, environment, evidence, root cause, fix, regression protection.

# 41. Clean code dan refactoring

Refactoring memperbaiki struktur tanpa mengubah behavior. Lakukan dengan target konkret: readability, testability, duplication, coupling, atau bug-prone flow.

## Tujuan belajar

- ☐ Menulis nama yang menggambarkan maksud.
- ☐ Menjaga function/controller fokus.
- ☐ Menghapus dead code dan magic value.
- ☐ Melakukan refactor dengan safety net.

## Materi inti

### Naming

Gunakan nama domain: submitComplaint lebih informatif daripada processData.

### Function

Pisahkan berdasarkan tanggung jawab, bukan sekadar jumlah baris.

### Duplication

Hapus duplikasi yang mewakili aturan sama; jangan abstraksikan dua kode yang kebetulan mirip tetapi semantiknya berbeda.

### Refactor safely

Test, small commits, diff review, performance/security re-check.

## Latihan praktik

1. Ambil satu feature lalu lakukan refactor dalam commit kecil terpisah.
2. Gunakan static analysis/linter bila tersedia dan pahami setiap perubahan.

### Checkpoint kompetensi

Diff refactor dapat dibaca, behavior tetap sama, dan alasan struktur baru dapat dijelaskan.

## 42. Materi lanjutan setelah roadmap inti

Setelah stack utama bisa sampai production, materi lanjutan dipilih berdasarkan masalah nyata. Jangan mempelajari semuanya sekaligus.

### Tujuan belajar

- ☐ Mengenali kapan Redis, realtime, search, object storage, message broker, OAuth/OIDC, multi-tenancy relevan.
- ☐ Memahami system design dan scaling secara bertahap.

### Materi inti

#### Data/infrastructure

Redis, object storage, CDN, database replication, search engine.

#### Async/distributed

Background jobs, message broker, event-driven architecture.

#### Identity

OAuth/OIDC untuk delegated auth/SSO.

#### Architecture

Multi-tenancy, load balancing, horizontal scaling, microservices setelah monolith dipahami.

#### Cloud/IaC

Cloud platform, Infrastructure as Code, advanced observability.

#### AI integration

Embeddings, agents, retrieval, evaluation, guardrails setelah application fundamentals kuat.

#### Checkpoint kompetensi

Pilih satu materi lanjutan karena requirement project, bukan karena sedang tren.

## 43. Yang tidak perlu dipelajari sekaligus

React, Vue, Next, Nuxt, Laravel, Go, Rust, Python, Docker, Kubernetes, dan cloud bukan daftar checklist serentak. Kedalaman pada satu stack sampai production lebih bernilai daripada mengenal banyak nama tanpa dapat menyelesaikan aplikasi.

### Tujuan belajar

- ☐ Membangun strategi fokus.
- ☐ Menghindari framework hopping.
- ☐ Mengukur kemampuan dengan proyek selesai.

### Materi inti

#### Core stack

PHP + Laravel + SQL + JavaScript/Vue/TypeScript cukup untuk banyak aplikasi production.

#### Specialization

Go ditambahkan setelah backend fundamental; Kubernetes hanya jika orchestration problem benar-benar muncul.

#### Transferable skills

HTTP, SQL, testing, security, Git, Linux, debugging, data modeling berpindah antar stack.

#### Checkpoint kompetensi

Punya daftar "sekarang", "nanti", dan "tidak perlu" untuk 3 bulan ke depan.

## 44. Portfolio yang menunjukkan kemampuan nyata

Portfolio teknis terbaik menunjukkan problem, keputusan, implementasi, quality, dan operasi - bukan hanya screenshot UI.

### Tujuan belajar

- ☐ Membuat project live/demo.
- ☐ Menulis README setup/architecture.
- ☐ Menyertakan schema, tests, security, performance, deployment notes.
- ☐ Menunjukkan commit history yang masuk akal.

### Materi inti

#### README

Problem, user, scope, stack, prerequisites, setup, env example, testing, deployment, architecture.

#### Evidence

Demo, screenshots secukupnya, schema/diagram, test output, CI, changelog.

#### Story

Jelaskan bug sulit, trade-off, keputusan security, performance improvement terukur.

#### Ethics

Jangan mempublikasikan real secret, private dataset, data warga, token, atau credential dalam portfolio.

### Latihan praktik

1. Ubah Portal Gampong menjadi case study 1-2 halaman.
2. Bersihkan repository dari secret/dummy sensitif.
3. Buat release v1.0 dengan changelog dan deployment notes.

#### Checkpoint kompetensi

Orang lain dapat clone, setup, menjalankan test, memahami arsitektur, dan melihat bukti bahwa aplikasi dipikirkan sampai production.

# BAGIAN IX

## Materi tambahan era digital 2026

### Arsitektur Full-Stack Modern

Authorization tetap di server; frontend tidak boleh menjadi sumber kebenaran izin.





## 45. AI-assisted development yang bertanggung jawab

AI dapat mempercepat ide, boilerplate, review, dokumentasi, dan debugging, tetapi output harus diperlakukan seperti code dari sumber eksternal: diverifikasi, diuji, dan dinilai keamanan serta lisensinya.

### Tujuan belajar

- ☐ Menggunakan AI sebagai pair assistant, bukan autopilot.
- ☐ Memberi context minimal yang aman.
- ☐ Memverifikasi output dengan docs/test.
- ☐ Menghindari membagikan secret/data sensitif.

### Materi inti

#### Use case

Menjelaskan error, membuat test case, merangkum docs, memberi alternatif desain, membuat draft migration, review diff.

#### Verification

Jalankan code, baca dependency/API versi, tulis test, cek security, cek edge case.

#### Privacy

Jangan paste password, token, private key, data personal, database dump, atau source yang tidak boleh dibagikan ke layanan eksternal.

#### Learning

Minta AI menjelaskan mental model dan memberi latihan; tetap tulis ulang dari nol untuk memastikan kamu benar-benar paham.

### Template prompt belajar

"Jelaskan konsep X dari mental model, beri contoh minimal, lalu beri 3 latihan bertingkat tanpa langsung memberi solusi. Setelah saya jawab, review alasan saya."

#### Checkpoint kompetensi

Setiap code dari AI dapat kamu jelaskan baris pentingnya, punya test/verifikasi, dan tidak membawa secret atau dependency misterius.

## 46. Software supply chain dan dependency hygiene

Aplikasi modern bergantung pada banyak package, image, action CI, dan layanan. Risiko tidak hanya datang dari kode sendiri tetapi juga rantai supply software.

### Tujuan belajar

- ☐ Mengunci dependency.
- ☐ Meminimalkan package.
- ☐ Memeriksa advisory/update.
- ☐ Menjaga CI permission dan provenance secara konseptual.

### Materi inti

#### Minimize

Setiap dependency menambah attack surface dan maintenance. Pilih package mapan hanya jika manfaatnya jelas.

#### Lock

Composer/npm lockfile membuat install reproducible.

#### Updates

Update kecil rutin lebih mudah direview daripada lonjakan besar. Baca changelog dan jalankan test.

#### CI

Pin action/version sesuai kebijakan, batasi token permission, lindungi secret, review third-party workflow.

### Latihan praktik

1. Audit dependency project dan hapus package tak terpakai.
2. Buat jadwal update dependency bulanan.
3. Periksa permission workflow CI.

#### Checkpoint kompetensi

Kamu tahu dependency utama, alasan dipakai, versi terkunci, dan cara mendeteksi update keamanan.

## 47. Privacy, data minimization, dan secure-by-design

Era digital membuat aplikasi mengumpulkan banyak data. Prinsip aman adalah mengumpulkan hanya yang diperlukan, membatasi akses, melindungi transit/storage, dan menetapkan retensi.

### Tujuan belajar

- ☐ Mengklasifikasi data.
- ☐ Menerapkan data minimization.
- ☐ Mendesain retention/deletion.
- ☐ Membatasi logging dan export.

### Materi inti

#### Minimization

Jika fitur tidak membutuhkan data tertentu, jangan minta dan simpan.

#### Access

Role bukan alasan memberi akses luas. Terapkan need-to-know dan audit action penting.

#### Retention

Data tidak harus disimpan selamanya. Tentukan lifecycle dan prosedur penghapusan sesuai konteks/aturan.

#### Logs

Log untuk operasi/security, bukan salinan database sensitif.

### Latihan praktik

1. Audit field form: wajib atau hanya "nice to have"?
2. Buat data classification sederhana: public/internal/confidential/sensitive.
3. Tentukan retention untuk log dan upload latihan.

#### Checkpoint kompetensi

Setiap field sensitif punya alasan, kontrol akses, retensi, dan tidak bocor lewat log/analytics.

## 48. Cloud fundamentals tanpa vendor lock-in awal

Cloud adalah model penyediaan compute, storage, database, network, observability, dan managed service. Pelajari konsep dulu sebelum menghafal nama produk vendor.

### Tujuan belajar

- ☐ Memahami compute/storage/database/network/CDN/secret manager.
- ☐ Memahami managed vs self-managed.
- ☐ Menghitung trade-off biaya, operasi, portability.
- ☐ Menerapkan environment separation.

### Materi inti

#### Compute

VM, container service, serverless/function adalah pilihan runtime dengan trade-off kontrol, startup, scaling, dan biaya.

#### Storage

Object storage berbeda dari filesystem lokal dan database.

#### Database

Managed database mengurangi operasi tetapi tetap membutuhkan backup, indexing, query design.

#### Environments

Pisahkan development/staging/production; secret dan dataset tidak dicampur.

### Latihan praktik

1. Petakan komponen project ke kategori cloud generik.
2. Buat diagram staging vs production.
3. Tulis risiko vendor lock-in dan kapan itu dapat diterima.

#### Checkpoint kompetensi

Bisa mendesain arsitektur cloud secara vendor-neutral sebelum memilih produk spesifik.

## 49. API economy, integrations, webhooks, dan idempotency

Aplikasi modern jarang berdiri sendiri. Integration perlu timeout, retry, signature verification, idempotency, mapping error, rate limit, dan observability.

### Tujuan belajar

- ☐ Mendesain client API yang tahan gagal.
- ☐ Memahami webhook verification secara konseptual.
- ☐ Menggunakan idempotency untuk duplicate delivery.
- ☐ Menjaga secret integration.

### Materi inti

#### Timeout

Jangan biarkan external call menunggu tanpa batas.

#### Retry

Retry hanya error transient dan gunakan backoff/jitter; jangan retry operation non-idempotent secara buta.

#### Webhook

Verifikasi signature/timestamp sesuai provider dan cegah replay sesuai dokumentasi.

#### Idempotency

Simpan event/idempotency key agar duplicate delivery tidak membuat efek ganda.

### Latihan praktik

1. Buat wrapper API dummy dengan timeout dan error mapping.
2. Simulasikan webhook yang dikirim dua kali dan pastikan hasil hanya sekali.

#### Checkpoint kompetensi

Integration tidak membuat request hang, duplicate side effect, atau mengekspos secret di frontend/log.

# 50. Dokumentasi, kolaborasi remote, dan engineering communication

Kemampuan teknis perlu dapat ditransfer ke tim. README, issue, PR, ADR, runbook, dan incident note mengurangi knowledge silo.

## Tujuan belajar

- ☐ Menulis issue yang dapat dikerjakan.
- ☐ Menulis PR yang mudah direview.
- ☐ Membuat ADR untuk keputusan arsitektur.
- ☐ Membuat runbook operasi.

## Materi inti

### Issue

Problem, scope, acceptance criteria, out-of-scope.

### PR

Summary, why, screenshots bila UI, test, migration, risk, rollback.

### ADR

Context, decision, alternatives, consequences.

### Async work

Tuliskan status dan blocker; jangan menyimpan keputusan hanya di chat pribadi.

## Latihan praktik

1. Tulis issue untuk fitur pengaduan.
2. Tulis ADR memilih Inertia vs SPA.
3. Tulis PR template untuk project.

### Checkpoint kompetensi

Orang lain bisa memahami perubahan dan keputusan tanpa harus bertanya ulang konteks dasar.

# 51. Karier belajar berkelanjutan di era teknologi cepat berubah

Framework berubah cepat; kemampuan menjaga skill tetap relevan berasal dari cara membaca dokumentasi, membangun project, mengevaluasi sumber, dan membuat loop belajar sendiri.

## Tujuan belajar

- ☐ Membaca dokumentasi resmi sebelum tutorial sekunder.
- ☐ Menggunakan release notes/changelog.
- ☐ Membuat learning backlog.
- ☐ Membedakan trend dari kebutuhan.

## Materi inti

### Official first

Dokumentasi resmi adalah sumber utama untuk API, versi, support, migration, dan security.

### Changelog

Baca perubahan breaking/deprecated sebelum upgrade.

### Learning backlog

Simpan pertanyaan dari project; prioritaskan yang membuka blocker atau meningkatkan quality.

### Portfolio loop

Build -> ship -> observe -> fix -> document -> repeat adalah loop belajar paling kuat.

#### Checkpoint kompetensi

Kamu tidak perlu mengetahui semua teknologi, tetapi tahu cara mempelajari teknologi baru secara sistematis ketika dibutuhkan.

# LAMPIRAN

Cheat sheet, mini-project code, rubric, glossary, referensi resmi

## Lampiran A - Cheat sheet command

| Area     | Command / pola                                                                                                                                 |
|----------|------------------------------------------------------------------------------------------------------------------------------------------------|
| Git      | git status   git diff   git switch -c feature/x   git add .   git commit -m "... "   git fetch   git pull --rebase   git log --oneline --graph |
| Composer | composer install   composer update <package>   composer dump-autoload   composer audit                                                         |
| Laravel  | php artisan about   route:list   migrate   test   queue:work   schedule:list   optimize:clear                                                  |
| npm/Vite | npm ci   npm run dev   npm run build   npm audit                                                                                               |
| Go       | go fmt ./...   go vet ./...   go test ./...   go test -race ./...   go run .   go build ./...                                                  |
| Linux    | pwd   ls -la   ps aux   ss -tulpn   df -h   free -h   tail -f <log>                                                                            |

## Lampiran B - Mini project lengkap: PHP CLI Expense Tracker

Tujuan: menggabungkan input-process-output, function, array, JSON file I/O, validation, dan error handling. Simpan sebagai expense.php lalu jalankan php expense.php.

### PHP

```
<?php
declare(strict_types=1);

const DATA_FILE = __DIR__ . '/expenses.json';

function loadExpenses(): array
{
    if (!is_file(DATA_FILE)) return [];
    $json = file_get_contents(DATA_FILE);
    $data = json_decode($json ? '[]', true);
    return is_array($data) ? $data : [];
}

function saveExpenses(array $items): void
{
    file_put_contents(
        DATA_FILE,
        json_encode($items, JSON_PRETTY_PRINT | JSON_UNESCAPED_UNICODE),
        LOCK_EX
    );
}

function addExpense(array &$items, string $name, int $amount): void
{
    $name = trim($name);
    if ($name === '' || $amount <= 0) {
        throw new InvalidArgumentException('Nama wajib dan amount > 0');
    }
    $items[] = [
        'id' => bin2hex(random_bytes(4)),
        'name' => $name,
```



```

        'amount' => $amount,
        'created_at' => date(DATE_ATOM),
    ];
}

function total(array $items): int
{
    return array_reduce($items, fn(int $sum, array $item) => $sum + (int)$item['amount'], 0);
}

$items = loadExpenses();
$command = $argv[1] ?? 'list';

try {
    if ($command === 'add') {
        $name = $argv[2] ?? '';
        $amount = (int)$argv[3] ?? 0;
        addExpense($items, $name, $amount);
        saveExpenses($items);
        echo "Tersimpan" . PHP_EOL;
    } elseif ($command === 'list') {
        foreach ($items as $item) {
            echo $item['id'] . " | " . $item['name'] . " | Rp" . number_format((int)
$item['amount'], 0, ',', '.') . PHP_EOL;
        }
        echo "TOTAL Rp" . number_format(total($items), 0, ',', '.') . PHP_EOL;
    } else {
        fwrite(STDERR, "Gunakan: add <nama> <amount> | list" . PHP_EOL);
        exit(2);
    }
} catch (Throwable $e) {
    fwrite(STDERR, $e->getMessage() . PHP_EOL);
    exit(1);
}

```

## Lampiran C - Mini project lengkap: kerangka modul Berita Laravel

Kode berikut menunjukkan vertical slice inti. Sesuaikan namespace, auth scaffolding, dan view layout proyekmu.

### PHP

```

// database/migrations/...create_posts_table.php
Schema::create('posts', function (Blueprint $table) {
    $table->id();
    $table->foreignId('user_id')->constrained()->cascadeOnDelete();
    $table->string('title', 200);
    $table->string('slug', 220)->unique();
    $table->text('body');
    $table->timestamp('published_at')->nullable()->index();
    $table->timestamps();
});

// app/Models/Post.php
class Post extends Model
{
    protected $fillable = ['title', 'slug', 'body', 'published_at'];
    protected function casts(): array { return ['published_at' => 'datetime']; }
    public function user(): BelongsTo { return $this->belongsTo(User::class); }
}

// app/Http/Requests/StorePostRequest.php

```

```
class StorePostRequest extends FormRequest
{
    public function authorize(): bool { return $this->user()->can('create', Post::class) ?? false; }
    public function rules(): array {
        return [
            'title' => ['required', 'string', 'max:200'],
            'body' => ['required', 'string', 'min:20'],
            'published_at' => ['nullable', 'date'],
        ];
    }
}

// app/Http/Controllers/PostController.php
class PostController extends Controller
{
    public function index(): View {
        $posts = Post::query()->with('user')->latest()->paginate(15);
        return view('posts.index', compact('posts'));
    }
    public function store(StorePostRequest $request): RedirectResponse {
        $post = $request->user()->posts()->create([
            ...$request->validated(),
            'slug' => Str::slug($request->string('title')) . '-' . Str::lower(Str::random(6)),
        ]);
        return redirect()->route('posts.show', $post);
    }
}

// routes/web.php
Route::middleware('auth')->group(function () {
    Route::resource('posts', PostController::class);
});
```

#### PHP

```
// tests/Feature/PostAuthorizationTest.php
public function test_user_cannot_edit_post_owned_by_other_user(): void
{
    $owner = User::factory()->create();
    $other = User::factory()->create();
    $post = Post::factory()->for($owner)->create();

    $this->actingAs($other)
        ->get(route('posts.edit', $post))
        ->assertForbidden();
}
```

## Lampiran D - Mini project lengkap: Go JSON health/items API

#### GO

```
package main

import (
    "encoding/json"
    "log"
    "net/http"
    "sync"
    "time"
)

type Item struct {
    ID    int    `json:"id"`
```

```

    Name string `json:"name"`
}

type Store struct {
    mu    sync.RWMutex
    next  int
    items []Item
}

func (s *Store) list(w http.ResponseWriter, r *http.Request) {
    s.mu.RLock()
    defer s.mu.RUnlock()
    w.Header().Set("Content-Type", "application/json")
    json.NewEncoder(w).Encode(map[string]any{"data": s.items})
}

func (s *Store) create(w http.ResponseWriter, r *http.Request) {
    var input struct{ Name string `json:"name"` }
    if err := json.NewDecoder(r.Body).Decode(&input); err != nil || input.Name == "" {
        http.Error(w, `{"error":"invalid input"}`, http.StatusBadRequest)
        return
    }
    s.mu.Lock()
    s.next++
    item := Item{ID: s.next, Name: input.Name}
    s.items = append(s.items, item)
    s.mu.Unlock()
    w.Header().Set("Content-Type", "application/json")
    w.WriteHeader(http.StatusCreated)
    json.NewEncoder(w).Encode(item)
}

func main() {
    store := &Store{}
    mux := http.NewServeMux()
    mux.HandleFunc("GET /health", func(w http.ResponseWriter, r *http.Request) {
        w.Header().Set("Content-Type", "application/json")
        json.NewEncoder(w).Encode(map[string]string{"status": "ok"})
    })
    mux.HandleFunc("GET /items", store.list)
    mux.HandleFunc("POST /items", store.create)

    srv := &http.Server{Addr: ":8080", Handler: mux, ReadHeaderTimeout: 5 * time.Second}
    log.Fatal(srv.ListenAndServe())
}

```

## Lampiran E - Rubric kompetensi

| Level                | Deskripsi                                                                         |
|----------------------|-----------------------------------------------------------------------------------|
| 0 - Belum            | Tidak dapat menjelaskan atau membuat contoh.                                      |
| 1 - Kenal            | Mengenali istilah tetapi butuh contoh/tutorial.                                   |
| 2 - Dasar            | Bisa membuat contoh kecil dengan dokumentasi.                                     |
| 3 - Mandiri          | Bisa membangun dan debug fitur dari requirement.                                  |
| 4 - Production-aware | Memikirkan test, security, performance, deploy, observability.                    |
| 5 - Dapat mengajar   | Bisa menjelaskan trade-off, review kode, dan membantu orang lain memahami konsep. |

## Lampiran F - Glossary inti

| Istilah | Definisi |
|---------|----------|
|---------|----------|

|                      |                                                                                 |
|----------------------|---------------------------------------------------------------------------------|
| API                  | Kontrak komunikasi antar software.                                              |
| Authentication       | Proses membuktikan identitas.                                                   |
| Authorization        | Proses menentukan izin.                                                         |
| Cache                | Salinan data untuk mempercepat akses.                                           |
| CI/CD                | Otomasi integrasi, test, build, delivery/deploy.                                |
| Component            | Bagian UI yang memiliki tanggung jawab dan interface/props.                     |
| Controller           | Boundary HTTP yang mengorkestrasi request-response.                             |
| CRUD                 | Create, Read, Update, Delete.                                                   |
| Dependency           | Kode/library/object yang dibutuhkan komponen lain.                              |
| DOM                  | Representasi tree dokumen HTML di browser.                                      |
| Environment variable | Konfigurasi di luar source code.                                                |
| Framework            | Kerangka kerja dengan konvensi dan fasilitas aplikasi.                          |
| HTTP                 | Protokol request-response web.                                                  |
| Idempotency          | Pengulangan operasi tidak menambah efek setelah hasil akhir yang sama tercapai. |
| Migration            | Perubahan schema database yang terversi.                                        |
| ORM                  | Pemetaan data relasional ke model/object.                                       |
| Queue                | Antrean pekerjaan asynchronous/background.                                      |
| RBAC                 | Authorization berdasarkan role.                                                 |
| REST                 | Gaya desain API berbasis resource dan semantics HTTP.                           |
| Session              | State server-side untuk interaksi user.                                         |
| Transaction          | Sekelompok operasi database yang commit/rollback bersama.                       |
| XSS                  | Eksekusi konten script tak terpercaya di browser.                               |
| CSRF                 | Pemaksaan request state-changing menggunakan sesi korban.                       |
| SQL Injection        | Input memengaruhi struktur query SQL.                                           |
| Observability        | Kemampuan memahami state sistem dari log, metrics, traces/events.               |
| SLO                  | Target reliabilitas yang didefinisikan untuk layanan.                           |
| Webhook              | HTTP callback dari sistem eksternal saat event terjadi.                         |
| Supply chain         | Rantai dependency/build/distribution software yang juga perlu diamankan.        |

## Lampiran G - Referensi resmi yang diprioritaskan

| Sumber                    | URL                                                                                                                                                       | Gunakan untuk                                                |
|---------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------|
| MDN Learn Web Development | <a href="https://developer.mozilla.org/en-US/docs/Learn_web_development">https://developer.mozilla.org/en-US/docs/Learn_web_development</a>               | HTML, CSS, JS, accessibility, web fundamentals               |
| PHP Manual / Downloads    | <a href="https://www.php.net/manual/">https://www.php.net/manual/</a>   <a href="https://www.php.net/downloads.php">https://www.php.net/downloads.php</a> | Bahasa PHP, standard library, versi security                 |
| Laravel 13 Documentation  | <a href="https://laravel.com/docs/13.x">https://laravel.com/docs/13.x</a>                                                                                 | Framework, Eloquent, validation, auth, queue, cache, testing |
| Laravel Release Notes     | <a href="https://laravel.com/docs/13.x/releases">https://laravel.com/docs/13.x/releases</a>                                                               | Support policy dan perubahan major                           |
| Vue 3 Guide               | <a href="https://vuejs.org/guide/">https://vuejs.org/guide/</a>                                                                                           | Vue 3 dan Composition API                                    |
| TypeScript Handbook       | <a href="https://www.typescriptlang.org/docs/handbook/">https://www.typescriptlang.org/docs/handbook/</a>                                                 | Type system dan language patterns                            |
| Go Documentation          | <a href="https://go.dev/doc/">https://go.dev/doc/</a>                                                                                                     | Tutorial, standard library, specification                    |
| Go 1.27 Release Notes     | <a href="https://go.dev/doc/go1.27">https://go.dev/doc/go1.27</a>                                                                                         | Perubahan Go 1.27                                            |
| Pro Git                   | <a href="https://git-scm.com/book/en/v2">https://git-scm.com/book/en/v2</a>                                                                               | Git dasar sampai workflow                                    |
| OWASP Top 10:2025         | <a href="https://top10.owasp.org/2025/en/">https://top10.owasp.org/2025/en/</a>                                                                           | Awareness risiko keamanan web terbaru                        |
| Docker Docs               | <a href="https://docs.docker.com/">https://docs.docker.com/</a>                                                                                           | Container, image, Compose                                    |
| Node.js Releases          | <a href="https://nodejs.org/en/about/previous-releases">https://nodejs.org/en/about/previous-releases</a>                                                 | Status Current/LTS/EOL                                       |
| GitHub Actions Docs       | <a href="https://docs.github.com/actions">https://docs.github.com/actions</a>                                                                             | CI/CD workflow                                               |
| web.dev                   | <a href="https://web.dev/">https://web.dev/</a>                                                                                                           | Performance dan Core Web Vitals                              |

### Cara memakai referensi

Gunakan dokumentasi yang sesuai versi project. Tutorial video membantu orientasi, tetapi ketika API berbeda atau error muncul, cek dokumentasi resmi, changelog, dan source/error message terlebih dahulu.

## Lampiran H - Checklist kelulusan roadmap

- ☐ Bisa bekerja dengan terminal, Git, branch, conflict, PR, dan secret hygiene.

- ☐ Bisa membuat program PHP CLI tanpa framework dan menjelaskan algoritma/data structure dasar.
- ☐ Bisa membuat halaman HTML/CSS responsive dan accessible serta JavaScript async/fetch.
- ☐ Bisa membaca HTTP request/response dan debug lewat DevTools.
- ☐ Bisa mendesain schema SQL, JOIN, transaction, index, dan menghindari N+1.
- ☐ Bisa membuat PHP web/PDO dan memahami session/password/upload sebelum Laravel.
- ☐ Bisa membuat Laravel CRUD terproteksi dengan validation, policy, relationship, test, upload, queue/cache/API.
- ☐ Bisa membuat Vue + TypeScript component dan full-stack Inertia module.
- ☐ Bisa melakukan security review, performance measurement, observability dasar.
- ☐ Bisa deploy staging dengan Linux/HTTPS/env/migration/queue/log/backup/rollback.
- ☐ Bisa menjalankan Docker dan CI pipeline dengan dependency/secret hygiene.
- ☐ Bisa membuat API Go kecil dan memahami concurrency setelah fondasi.
- ☐ Bisa menggunakan AI sebagai assistant dengan verifikasi dan privasi.
- ☐ Punya portfolio project yang dapat dijalankan, diuji, dan dijelaskan end-to-end.

Jika sebagian besar checklist ini dapat dibuktikan dengan project, test, deployment, dan penjelasan mandiri, kamu sudah memiliki fondasi full-stack yang kuat dan siap memperdalam bidang tertentu: backend, frontend, cloud, security, data, atau Go.